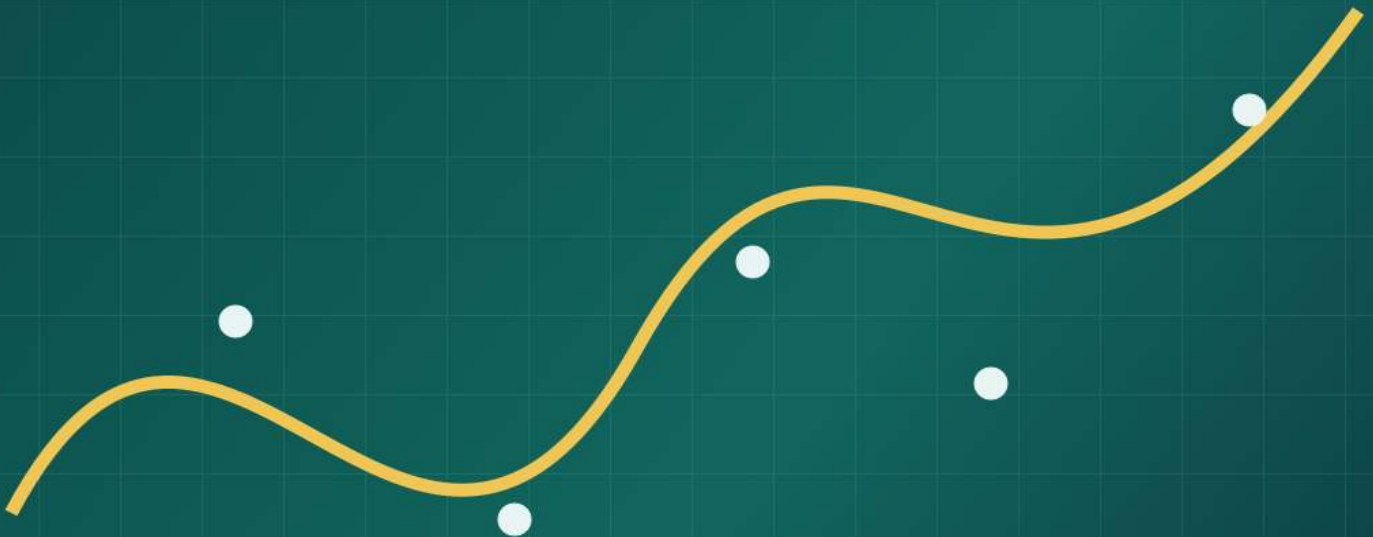


LIBRO ABIERTO · EDICIÓN INTERACTIVA



Álgebra Superior con R y GeoGebra

Teoría, visualización y experimentación



Jesús Gilberto Rodríguez Escobedo

Primera edición digital · San Luis Potosí, México · 2026

MANUAL → R → GEOGEBRA → INTERPRETACIÓN

Tabla de contenidos

Bienvenida	9
Créditos editoriales	10
Licencia y uso académico	12
Prefacio	13
Agradecimientos	14
Presentación	15
Introducción	16
Notación general	17
1 Lógica y conjuntos	18
1.1 Lenguaje de conjuntos	19
1.2 Proposiciones y conectivos lógicos	21
1.3 Cuantificadores y equivalencias lógicas	22
1.4 Operaciones y propiedades de los conjuntos	24
1.5 Cardinalidad de conjuntos finitos y problemas de conteo	26
1.6 Producto cartesiano y relaciones	27
1.7 Funciones: definición, representación y operaciones	30
1.8 Funciones inyectivas, suprayectivas y biyectivas	32
1.9 Cardinalidad, equipotencia y conjuntos infinitos	34
2 Números enteros y reales	43
2.1 Propiedades de los números enteros	44
2.2 Inducción matemática	45
2.3 Divisibilidad	47
2.4 Números primos y factorización	50
2.5 Números racionales y números reales	53
2.6 Propiedades de los números reales	54
2.7 Exponentes racionales y negativos	56

2.8	Valor absoluto	58
3	Números complejos	62
3.1	Definición de $\mathbb{R}^2$	62
3.2	Representaciones cartesiana y polar de vectores en $\mathbb{R}^2$	64
3.3	Operaciones con vectores en $\mathbb{R}^2$	66
3.4	Módulo y argumento	68
3.5	Números imaginarios y complejos	69
3.6	Operaciones básicas con números complejos	71
3.7	Complejo conjugado y sus propiedades	72
3.8	División de complejos	73
3.9	Potencias y raíces de complejos	75
4	Polinomios	79
4.1	Definición de polinomio	79
4.2	Aritmética y propiedades de los polinomios	81
4.3	Divisibilidad	82
4.4	Definición de raíz de un polinomio	83
4.5	Teorema del residuo y división sintética	84
4.6	Obtención de raíces múltiples	85
4.7	Teorema fundamental del álgebra	86
4.8	Descomposición de un polinomio en factores lineales	87
4.9	Propiedades de polinomios con coeficientes reales	88
4.10	Funciones racionales	91
4.11	Fracciones parciales	92
5	Métodos numéricos para la estimación de raíces	101
5.1	Acotación de raíces	103
5.2	Separación de raíces	105
5.3	Teorema de Sturm	106
5.4	Regla de los signos de Descartes	109
5.5	Estimación mediante bisección	110
5.6	Estimación mediante el método de la secante	112
5.7	Estimación mediante el método de Newton	113
	Glosario	119
	Índices de consulta	121
	Índice temático	121
	Índice de funciones y operadores de R	122
	Índice de símbolos	124
	Índice de figuras	125

Índice de laboratorios	126
Referencias generales	128
Acerca del autor	130

Listado de Figuras

1.1	Cuatro operaciones representadas mediante regiones de Venn: unión, intersección, complemento y diferencia simétrica.	25
1.2	Una relación general, una función válida y una correspondencia que no cumple la definición de función.	30
1.3	Recorrido de un elemento mediante f , después mediante g y directamente mediante la composición $g \circ f$	32
1.4	La inyectividad evita imágenes repetidas; la suprayectividad exige cubrir todo el codominio; la biyección cumple ambas condiciones.	33
1.5	Implementación de $X = (A + \overline{B})C$ con inversor, compuerta OR y compuerta AND.	36
1.6	Las señales se dividen en intervalos constantes; en cada uno se evalúa $X = (A + \overline{B})C$	40
2.1	La suma acumulada de impares coincide con los cuadrados perfectos.	46
2.2	Los pares sucesivos del algoritmo de Euclides terminan en el máximo común divisor.	49
2.3	Cadena de factorización y descomposición prima de 756.	52
2.4	Los naturales, enteros, racionales e irracionales se sitúan dentro de los reales.	54
2.5	La desigualdad $ x - a \leq r$ selecciona un intervalo centrado en a	58
3.1	Del par cartesiano $(3, 4)$ a radio 5 y argumento θ	65
3.2	La diagonal representa $\mathbf{u} + \mathbf{v}$	67
3.3	Conjugar refleja un punto respecto del eje real.	72
3.4	Las raíces sextas de la unidad son los vértices de un hexágono regular.	76
4.1	La división polinomial produce un cociente y un residuo de grado menor que el divisor.	83
4.2	Comportamiento local de raíces de multiplicidad uno, dos y tres.	86
4.3	Las raíces no reales de un polinomio real son simétricas respecto del eje real.	90
4.4	Una función racional puede presentar huecos y asíntotas determinadas por sus factores.	92
5.1	La cota global contiene todas las raíces; la malla y los cambios de signo producen intervalos de búsqueda más pequeños.	104

5.2	Las primeras bisecciones reducen sistemáticamente el intervalo que encierra la raíz.	112
5.3	La secante usa una cuerda definida por dos puntos; Newton usa una tangente y requiere la derivada.	115

Listado de Tablas

5.1	Índice alfabético de conceptos, código y laboratorios.	121
5.2	Funciones y operadores principales de R.	122
5.3	Símbolos matemáticos principales.	124
5.4	Laboratorios interactivos incorporados.	126

Portada

Jesús Gilberto Rodríguez Escobedo

Primera edición digital, 2026 · San Luis Potosí, México

Libro abierto bajo licencia CC BY-NC-SA 4.0.

Bienvenida

Bienvenido a *Álgebra Superior con aplicaciones en R y GeoGebra*. Este libro propone aprender mediante cuatro acciones coordinadas: resolver manualmente, implementar en R, visualizar en GeoGebra e interpretar el resultado.

La computadora no reemplaza el razonamiento matemático. Se utiliza para experimentar, comprobar conjeturas, comparar métodos y estudiar situaciones difíciles de explorar sólo con lápiz y papel. Cada recurso debe responder una pregunta matemática clara.

Objetivos generales

- comprender los conceptos esenciales del Álgebra Superior;
- traducir definiciones y procedimientos a algoritmos reproducibles;
- utilizar R para calcular, organizar y verificar resultados;
- emplear GeoGebra para visualizar relaciones, funciones, números complejos, polinomios y raíces;
- comunicar conclusiones con notación, gráficas e interpretación correctas.

El volumen se coordina con *Fundamentos Matemáticos del Álgebra Superior*, donde se desarrollan con mayor profundidad las definiciones, propiedades y demostraciones.

Créditos editoriales

Título: *Álgebra Superior con aplicaciones en R y GeoGebra*

Subtítulo: Teoría, visualización y experimentación

Autor: Jesús Gilberto Rodríguez Escobedo

Edición: Primera edición digital, 2026

Revisión: 14 de agosto de 2026

Lugar de edición: San Luis Potosí, México

Producción editorial: edición académica abierta desarrollada con Quarto, R, GeoGebra, Jupyter, Git y GitHub Pages.

Formatos: libro web interactivo, PDF descargable, cuadernos y construcciones .ggb.

Cómo citar esta obra

Rodríguez Escobedo, J. G. (2026). *Álgebra Superior con aplicaciones en R y GeoGebra: teoría, visualización y experimentación*. Primera edición digital. San Luis Potosí, México.
<https://gilbertorodriguez59.github.io/libro-algebra-superior-r-geogebra/>

Entrada BibTeX sugerida

```
@book{rodriguez2026algebrargeogebra,  
  author    = {Rodríguez Escobedo, Jesús Gilberto},  
  title     = {Álgebra Superior con aplicaciones en R y GeoGebra},  
  subtitle  = {Teoría, visualización y experimentación},  
  year      = {2026},  
  address   = {San Luis Potosí, México},  
  note      = {Primera edición digital, libro web interactivo y PDF; revisión del 14 de ago},  
  url       = {https://gilbertorodriguez59.github.io/libro-algebra-superior-r-geogebra/}  
}
```

Volumen teórico complementario

Las definiciones, propiedades y demostraciones se amplían en *Fundamentos Matemáticos del Álgebra Superior*.

Licencia y uso académico

Esta obra se distribuye bajo la licencia [Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional \(CC BY-NC-SA 4.0\)](https://creativecommons.org/licenses/by-nc-sa/4.0/).

Puede copiarse, redistribuirse y adaptarse con fines académicos siempre que se reconozca la autoría, no exista uso comercial sin autorización y las obras derivadas se compartan bajo la misma licencia.

R es software libre y GeoGebra se utiliza conforme a sus condiciones de licencia. Los nombres, marcas y plataformas pertenecen a sus respectivos titulares. Los cuadernos, scripts y construcciones pueden modificarse para actividades educativas conservando la atribución y la licencia de esta obra.

Uso responsable de recursos computacionales

El código y los laboratorios se proporcionan con fines educativos. El lector debe revisar datos de entrada, supuestos, tolerancias y resultados antes de reutilizarlos. Una salida numérica o gráfica no constituye por sí sola una demostración matemática.

Prefacio

Este libro surge de una experiencia reiterada en el aula: muchos estudiantes comprenden mejor una idea algebraica cuando pueden verla, modificarla y comprobarla, pero el aprendizaje pierde profundidad si la exploración computacional no se conecta con definiciones y razonamientos precisos.

Por ello, cada tema busca articular cuatro momentos: desarrollo manual, traducción a R, exploración visual en GeoGebra e interpretación. El propósito no es enseñar comandos aislados, sino mostrar cómo una pregunta matemática se transforma en un procedimiento reproducible y en una representación que ayuda a pensar.

El libro acompaña al volumen *Fundamentos Matemáticos del Álgebra Superior*. Mientras aquel profundiza en conceptos y demostraciones, éste concentra la experimentación y las aplicaciones. Juntos permiten pasar de la intuición a la formalización, y de la formalización a la comprobación computacional.

Espero que los cuadernos, construcciones y actividades ayuden al lector a desarrollar autonomía: formular una conjetura, diseñar una prueba, detectar un error y explicar con claridad lo que observa.

Jesús Gilberto Rodríguez Escobedo
San Luis Potosí, México

Agradecimientos

Agradezco a los estudiantes que, durante décadas de docencia, han mostrado con sus preguntas qué conceptos requieren otra explicación, una figura más clara o un experimento mejor diseñado.

Reconozco a los colegas e instituciones con quienes he compartido proyectos de matemáticas, computación, inteligencia artificial y tecnología educativa. Agradezco asimismo a las comunidades de R, Quarto, Jupyter y GeoGebra, cuyas herramientas hacen posible crear materiales abiertos y reproducibles.

Mi gratitud final es para mi familia, por su apoyo y paciencia durante la elaboración de esta obra.

Presentación

Álgebra Superior con aplicaciones en R y GeoGebra integra teoría, cálculo, visualización y experimentación. Está dirigido a estudiantes y profesores de matemáticas, ingeniería, computación y ciencias aplicadas que desean comprender los conceptos y, al mismo tiempo, explorar su comportamiento mediante herramientas digitales.

Ruta didáctica

1. **Comprensión manual:** definición, ejemplo y procedimiento paso a paso.
2. **Implementación en R:** cálculo reproducible y verificación numérica.
3. **Visualización en GeoGebra:** exploración dinámica de parámetros y representaciones.
4. **Interpretación:** explicación matemática de los resultados, alcances y errores.

Estructura actual

Los cinco capítulos estudian lógica y conjuntos; enteros y reales; números complejos; polinomios; y métodos numéricos para raíces. Cada uno enlaza recursos descargables pertinentes. Los índices, el glosario y las referencias permiten utilizar la obra también como material de consulta.

Introducción

El Álgebra Superior proporciona el lenguaje con el que se describen estructuras, operaciones, funciones y métodos de solución. R y GeoGebra permiten ampliar ese estudio: el primero facilita programar cálculos y comparar algoritmos; el segundo convierte relaciones y variaciones en objetos visuales manipulables.

La integración de estas herramientas tiene sentido cuando se conserva el orden conceptual. Primero se identifica el problema y su notación; después se desarrolla un procedimiento manual; luego se implementa o visualiza; finalmente se interpreta el resultado y se contrasta con la teoría.

Los cuadernos de R se diseñan para ejecutarse paso a paso. Las construcciones de GeoGebra permiten modificar valores y observar invariantes o cambios. En ambos casos, el lector debe registrar qué modifica, qué permanece y qué explicación matemática justifica el comportamiento observado.

La versión web contiene los enlaces y laboratorios interactivos; el PDF conserva la exposición, el código y las indicaciones para acceder a los recursos descargables.

Notación general

Notación	Significado general
A, B, U	conjuntos; U suele ser el conjunto universal
$x \in A, A \subseteq B$	pertenencia e inclusión
$\forall, \exists, \neg, \wedge, \vee$	cuantificadores y conectivos
$\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{C}$	sistemas numéricos
$f : A \rightarrow B$	función de A en B
$z = a + bi$	número complejo
$p(x), \deg p$	polinomio y su grado
x_k	aproximación en la iteración k
ε	tolerancia numérica
$<-$	asignación en R
$\mathbf{f}(\mathbf{x})$	llamada de una función en R

En el texto matemático se usa $=$ para igualdad. En R, $<-$ asigna un valor a un objeto y $==$ compara valores. Esta diferencia evita confundir una afirmación matemática con una instrucción de programa.

1 Lógica y conjuntos

Experimentación con R, GeoGebra y circuitos

Introducción

Este capítulo transforma los conceptos de lógica y conjuntos en objetos que pueden calcularse, visualizarse y comprobarse. Cada bloque parte de una idea matemática, la resuelve manualmente y después utiliza R como laboratorio. GeoGebra se reservará para las representaciones dinámicas y Shiny para actividades en las que el estudiante modifique datos y reciba retroalimentación.

El código inicial utiliza R base (R Core Team 2026). De esta forma puede ejecutarse en RStudio o en el cuaderno de Google Colab sin instalar paquetes.

* Cuaderno completo y ejecutable

El cuaderno reúne las nueve secciones, las aplicaciones a sistemas digitales, visualizaciones en R y ejercicios de comprobación. Puede abrirse en Colab o descargarse para conservar una copia local.

[Abrir en Google Colab](#) [Descargar el cuaderno](#)

Objetivos

Al finalizar, el lector podrá:

- representar conjuntos finitos mediante vectores de R;
- construir tablas de verdad;
- evaluar proposiciones abiertas sobre dominios finitos;
- comprobar equivalencias lógicas y leyes de conjuntos;
- resolver problemas de inclusión-exclusión;
- representar productos cartesianos, relaciones y funciones;
- clasificar funciones finitas;

- experimentar con correspondencias biyectivas;
- comprobar simplificaciones booleanas y diseñar circuitos.

1.1 Lenguaje de conjuntos

Situación inicial

Consideremos el universo

$$U = \{1, 2, \dots, 10\},$$

el conjunto A de números pares y el conjunto B de números primos en U :

$$A = \{2, 4, 6, 8, 10\}, \quad B = \{2, 3, 5, 7\}.$$

En R, un conjunto finito puede representarse mediante un vector sin elementos repetidos.

```
U <- 1:10
A <- c(2, 4, 6, 8, 10)
B <- c(2, 3, 5, 7)

A
B
```

El operador `%in%` comprueba pertenencia.

```
4 %in% A
5 %in% A
A %in% B
```

Para comprobar $A \subseteq B$, no basta que algún elemento pertenezca; todos deben pertenecer:

```
es_subconjunto <- function(X, Y) {
  all(X %in% Y)
}

es_subconjunto(c(2, 4), A)
es_subconjunto(A, B)
```

La igualdad de conjuntos debe ignorar el orden:

```
setequal(c(1, 2, 3), c(3, 2, 1))
identical(c(1, 2, 3), c(3, 2, 1))
```

La función **setequal()** devuelve verdadero, mientras que **identical()** devuelve falso, porque esta última compara vectores ordenados, no conjuntos.

Conjunto potencia

El siguiente código genera todos los subconjuntos de un conjunto finito:

```
conjunto_potencia <- function(A) {
  resultado <- list(A[FALSE])
  if (length(A) > 0) {
    for (k in seq_len(length(A))) {
      resultado <- c(
        resultado,
        combn(A, k, simplify = FALSE)
      )
    }
  }
  resultado
}

P <- conjunto_potencia(c("a", "b", "c"))
P
length(P)
```

El resultado contiene $2^3 = 8$ subconjuntos, incluido el conjunto vacío.

* Actividad con el laboratorio publicado

Use la salida de **conjunto_potencia()** para elegir dos subconjuntos de $\{a, b, c\}$. Después, compruebe su unión, intersección y diferencia en el [laboratorio de operaciones con conjuntos](#). Así se coordinan la enumeración en R y la representación mediante regiones de Venn.

1.2 Proposiciones y conectivos lógicos

Construcción de una tabla de verdad

R representa verdadero y falso mediante **TRUE** y **FALSE**. Las combinaciones para dos proposiciones se generan así:

```
tabla <- expand.grid(
  q = c(TRUE, FALSE),
  p = c(TRUE, FALSE)
)[, c("p", "q")]

implica <- function(p, q) {
  (!p) | q
}

tabla$no_p <- !tabla$p
tabla$p_y_q <- tabla$p & tabla$q
tabla$p_o_q <- tabla$p | tabla$q
tabla$p_implica_q <- implica(tabla$p, tabla$q)
tabla$p_sii_q <- tabla$p == tabla$q

tabla
```

La implicación se implementa mediante la equivalencia

$$p \Rightarrow q \equiv \neg p \vee q.$$

Tautologías, contradicciones y contingencias

Una expresión es tautología si todas sus filas son verdaderas:

```
es_tautologia <- function(valores) all(valores)
es_contradiccio <- function(valores) all(!valores)

tercero_excluido <- tabla$p | !tabla$p
contradiccio <- tabla$p & !tabla$p
```

```
es_tautologia(tercero_excluido)
es_contradiccion(contradiccion)
```

Para comprobar

$$p \Rightarrow (q \wedge r) \equiv (p \Rightarrow q) \wedge (p \Rightarrow r),$$

generamos las ocho combinaciones posibles:

```
t3 <- expand.grid(
  r = c(TRUE, FALSE),
  q = c(TRUE, FALSE),
  p = c(TRUE, FALSE)
)[, c("p", "q", "r")]

izquierda <- implica(t3$p, t3$q & t3$r)
derecha <- implica(t3$p, t3$q) & implica(t3$p, t3$r)

data.frame(t3, izquierda, derecha, equivalentes = izquierda == derecha)
all(izquierda == derecha)
```

La última instrucción devuelve verdadero; la equivalencia se cumple en todas las filas.

Interactivo: tabla de verdad

Cambie la expresión y los valores de entrada. La fila activa se señala en la tabla y la salida se actualiza inmediatamente.

[Abrir el laboratorio de tablas de verdad](#)

1.3 Cuantificadores y equivalencias lógicas

Proposiciones abiertas sobre dominios finitos

Sea

$$D = \{-4, -3, \dots, 4\}$$

y sea $p(x) : x^2 \leq 4$. Su conjunto solución se obtiene filtrando el dominio:

```
D <- -4:4
p <- D^2 <= 4

data.frame(x = D, p_x = p)
D[p]
```

El conjunto solución es $\{-2, -1, 0, 1, 2\}$.

En un dominio finito, **all()** representa el cuantificador universal y **any()** el existencial:

```
all(D^2 >= 0) # para todo x en D, x^2 >= 0
any(D^2 == 4) # existe x en D tal que x^2 = 4
```

! Límite de la comprobación computacional

Verificar una propiedad en muchos valores no demuestra que se cumpla para todos los números reales. R sí puede decidir un cuantificador sobre un dominio finito completamente enumerado. Para dominios infinitos, la demostración matemática sigue siendo indispensable.

Negación de cuantificadores

La equivalencia

$$\neg(\forall x \in D p(x)) \equiv \exists x \in D \neg p(x)$$

se comprueba con:

```
lado_izquierdo <- !all(p)
lado_derecho <- any(!p)

c(lado_izquierdo, lado_derecho)
```

No estamos sustituyendo una demostración formal; estamos observando la equivalencia en el dominio elegido.

1.4 Operaciones y propiedades de los conjuntos

Operaciones en R

R incluye funciones directas para unión, intersección y diferencia.

```
union(A, B)
intersect(A, B)
setdiff(A, B)
setdiff(B, A)
```

El complemento depende del universo:

```
complemento <- function(X, U) {
  setdiff(U, X)
}

complemento(A, U)
```

La diferencia simétrica es

$$A \triangle B = (A \setminus B) \cup (B \setminus A).$$

```
diferencia_simetrica <- function(X, Y) {
  union(setdiff(X, Y), setdiff(Y, X))
}

diferencia_simetrica(A, B)
```

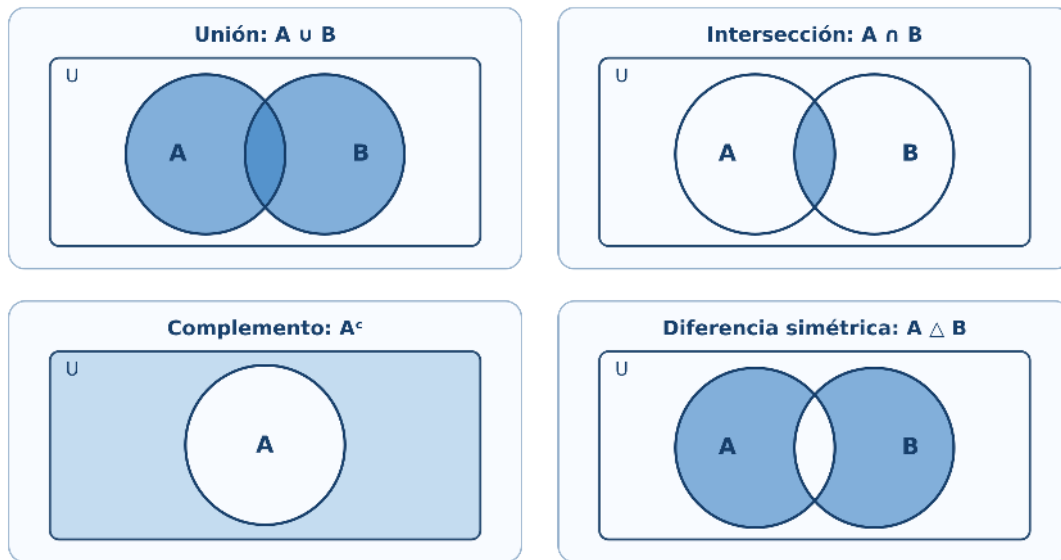



Figura 1.1: Cuatro operaciones representadas mediante regiones de Venn: unión, intersección, complemento y diferencia simétrica.

Interactivo: operaciones de conjuntos

Seleccione una operación y modifique los elementos de A y B . El laboratorio calcula el conjunto resultante, su cardinalidad y la región correspondiente.

[Abrir el laboratorio de conjuntos](#)

Comprobación de las leyes de De Morgan

Para los conjuntos concretos $A, B \subseteq U$, comprobamos

$$(A \cup B)^c = A^c \cap B^c.$$

```
lado_izquierdo <- complemento(union(A, B), U)
lado_derecho <- intersect(
  complemento(A, U),
  complemento(B, U)
)
```

```
lado_izquierdo
lado_derecho
setequal(lado_izquierdo, lado_derecho)
```

i Del interactivo a la demostración

El laboratorio permite anticipar que $(A \cup B)^c$ y $A^c \cap B^c$ somborean la misma región. La comprobación con R verifica el universo finito elegido; la demostración general se realiza mediante pertenencia.

1.5 Cardinalidad de conjuntos finitos y problemas de conteo

Inclusión-exclusión con dos conjuntos

En un grupo de 180 estudiantes, 95 usan R, 80 usan Python y 40 utilizan ambos lenguajes.

```
total <- 180
n_R <- 95
n_Python <- 80
n_ambos <- 40

n_al_menos_uno <- n_R + n_Python - n_ambos
n_ninguno <- total - n_al_menos_uno

c(al_menos_uno = n_al_menos_uno, ninguno = n_ninguno)
```

Inclusión-exclusión con tres conjuntos

```
inclusion_exclusion_3 <- function(nA, nB, nC,
                                nAB, nAC, nBC, nABC) {
  nA + nB + nC - nAB - nAC - nBC + nABC
}

total_dispositivos <- 400
al_menos_un_sensor <- inclusion_exclusion_3(
  nA = 170, nB = 150, nC = 140,
```

```

  nAB = 70, nAC = 60, nBC = 50,
  nABC = 20
)

ningun_sensor <- total_dispositivos - al_menos_un_sensor
c(al_menos_un_sensor, ningun_sensor)

```

Interpretación

R no decide qué significa cada intersección. El lector debe identificar si los datos de dos conjuntos incluyen o excluyen la intersección triple. El código ejecuta correctamente una fórmula solo cuando la modelación es correcta.

1.6 Producto cartesiano y relaciones

Producto cartesiano como tabla

Sean $A = \{1, 2\}$ y $B = \{x, y, z\}$. El producto $A \times B$ se genera con **expand.grid()**:

```

A1 <- c(1, 2)
B1 <- c("x", "y", "z")

AxB <- expand.grid(a = A1, b = B1)
AxB
nrow(AxB)

```

El número de filas confirma $|A \times B| = |A||B| = 6$.

Comprobación de una identidad

Verifiquemos

$$A \times (B \cup C) = (A \times B) \cup (A \times C).$$

Para comparar pares ordenados utilizaremos una clave de texto.

```

clave_par <- function(x, y) paste(x, y, sep = "|")

producto_cartesiano <- function(X, Y) {
  pares <- expand.grid(x = X, y = Y)
  clave_par(pares$x, pares$y)
}

A2 <- c(1, 2)
B2 <- c("b1", "b2")
C2 <- c("c1", "c2")

izquierda <- producto_cartesiano(A2, union(B2, C2))
derecha <- union(
  producto_cartesiano(A2, B2),
  producto_cartesiano(A2, C2)
)

setequal(izquierda, derecha)

```

Propiedades de una relación finita

Representemos una relación mediante dos columnas:

```

X <- c(1, 2, 3)
R <- data.frame(
  x = c(1, 1, 2, 3),
  y = c(1, 2, 2, 3)
)
R

```

Las siguientes funciones analizan reflexividad, simetría, antisimetría y transitividad.

```

claves_relacion <- function(R) clave_par(R$x, R$y)

es_reflexiva <- function(R, X) {
  all(clave_par(X, X) %in% claves_relacion(R))
}

es_simetrica <- function(R) {

```

```

    all(clave_par(R$y, R$x) %in% claves_relacion(R))
}

es_antisimetrica <- function(R) {
  reversos <- clave_par(R$y, R$x) %in% claves_relacion(R)
  all((R$x == R$y) | !reversos)
}

es_transitiva <- function(R) {
  claves <- claves_relacion(R)
  for (i in seq_len(nrow(R))) {
    for (j in seq_len(nrow(R))) {
      if (R$y[i] == R$x[j]) {
        if (!(clave_par(R$x[i], R$y[j]) %in% claves)) {
          return(FALSE)
        }
      }
    }
  }
  TRUE
}

c(
  reflexiva = es_reflexiva(R, X),
  simetrica = es_simetrica(R),
  antisimetrica = es_antisimetrica(R),
  transitiva = es_transitiva(R)
)

```

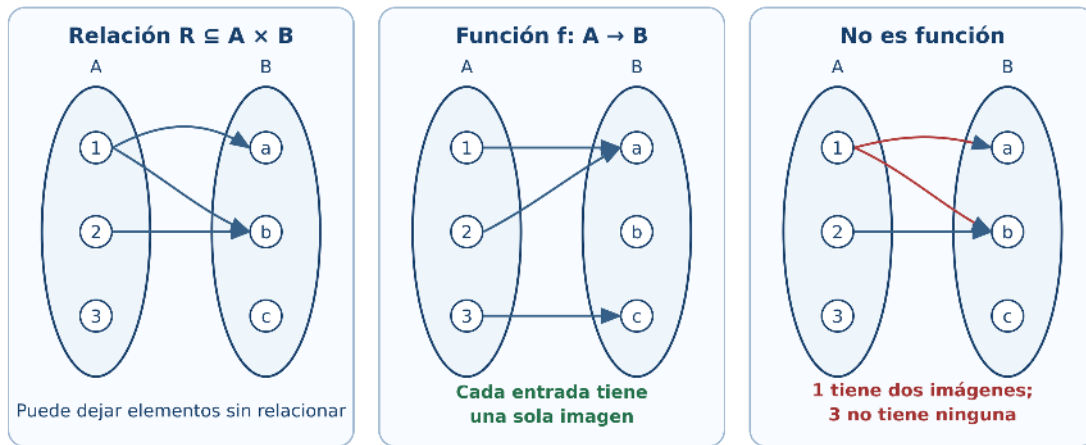


Figura 1.2: Una relación general, una función válida y una correspondencia que no cumple la definición de función.

* Interpretación

El programa comprueba una relación finita completamente especificada. La razón matemática de cada propiedad sigue siendo la definición; el código automatiza la revisión de los pares.

1.7 Funciones: definición, representación y operaciones

Una función como tabla

Sobre conjuntos finitos, una función puede representarse por una tabla de entradas y salidas.

```
dominio <- c("s1", "s2", "s3")
codominio <- c("bajo", "medio", "alto")

f_tabla <- data.frame(
  x = dominio,
  y = c("bajo", "alto", "medio")
)
```

```
f_tabla
```

Una tabla representa una función si:

1. cada elemento del dominio aparece;
2. aparece exactamente una vez como entrada;
3. cada salida pertenece al codominio.

```
es_funcion <- function(tabla, dominio, codominio) {  
  entradas_correctas <- setequal(tabla$x, dominio)  
  entradas_unicas <- !anyDuplicated(tabla$x)  
  salidas_validas <- all(tabla$y %in% codominio)  
  
  entradas_correctas && entradas_unicas && salidas_validas  
}  
  
es_funcion(f_tabla, dominio, codominio)
```

Funciones dadas por fórmulas

```
f <- function(x) 2 * x + 1  
g <- function(x) x^2  
  
g_comp_f <- function(x) g(f(x))  
f_comp_g <- function(x) f(g(x))  
  
x <- -3:3  
data.frame(  
  x,  
  g_comp_f = g_comp_f(x),  
  f_comp_g = f_comp_g(x)  
)
```

Las columnas no coinciden: en general $g \circ f \neq f \circ g$.

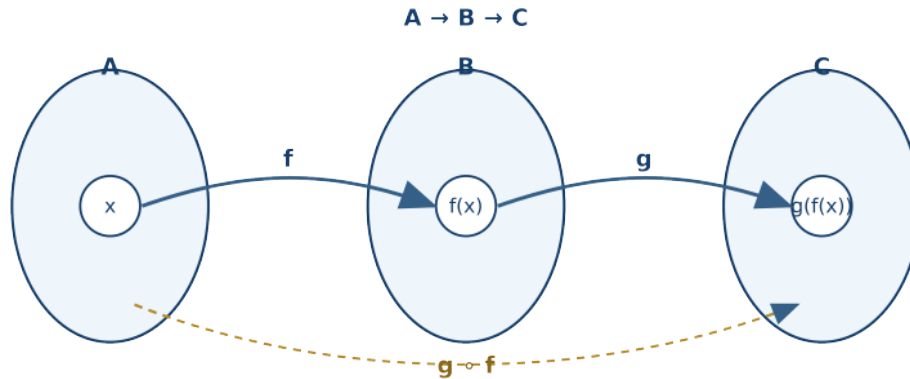


Figura 1.3: Recorrido de un elemento mediante f , después mediante g y directamente mediante la composición $g \circ f$.

*** Actividad de exploración**

Cambie las definiciones de f y g , calcule ambas composiciones sobre el mismo vector x y use la figura anterior para seguir un valor desde el dominio hasta la salida. Identifique después condiciones particulares bajo las cuales $g \circ f = f \circ g$.

1.8 Funciones inyectivas, suprayectivas y biyectivas

Clasificación automática de funciones finitas

```
clasificar_funcion <- function(tabla, dominio, codominio) {
  if (!es_funcion(tabla, dominio, codominio)) {
    return(data.frame(
      es_funcion = FALSE,
      inyectiva = NA,
      suprayectiva = NA,
      biyectiva = NA
    ))
  }

  inyectiva <- !anyDuplicated(tabla$y)
  suprayectiva <- setequal(unique(tabla$y), codominio)
```



```

data.frame(
  es_funcion = TRUE,
  inyectiva = inyectiva,
  suprayectiva = suprayectiva,
  biyectiva = inyectiva && suprayectiva
)
}

clasificar_funcion(f_tabla, dominio, codominio)

```

Probemos ahora una función que repite una salida:

```

h_tabla <- data.frame(
  x = dominio,
  y = c("bajo", "bajo", "medio")
)

clasificar_funcion(h_tabla, dominio, codominio)

```

No es inyectiva porque dos entradas producen “bajo”, y no es suprayectiva porque “alto” no aparece como imagen.

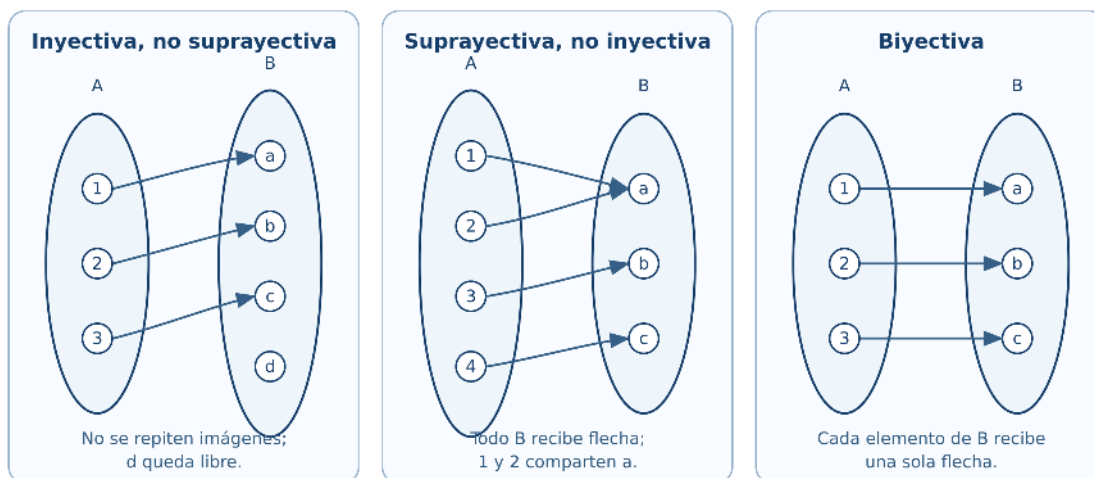


Figura 1.4: La inyectividad evita imágenes repetidas; la suprayectividad exige cubrir todo el codominio; la biyección cumple ambas condiciones.

Interactivo: clasificador de funciones

Modifique las tres imágenes, deje alguna entrada sin imagen o asigne dos imágenes al elemento 1. El diagnóstico distingue la definición de función de las propiedades de inyectividad, suprayectividad y biyección.

[Abrir el clasificador de funciones](#)

1.9 Cardinalidad, equipotencia y conjuntos infinitos

Una biyección entre $\mathbb{N}_0$ y $\mathbb{Z}$

Definimos

$$\phi(n) = \begin{cases} 0, & n = 0, \\ (n+1)/2, & n \text{ impar}, \\ -n/2, & n \text{ par y } n > 0. \end{cases}$$

```
phi <- function(n) {  
  ifelse(  
    n == 0,  
    0,  
    ifelse(n %% 2 == 1, (n + 1) / 2, -n / 2)  
  )  
}  
  
n <- 0:16  
data.frame(n, phi_n = phi(n))
```

La salida comienza

0, 1, -1, 2, -2, 3, -3, ...

y muestra cómo los enteros pueden enumerarse mediante los naturales.

! Qué demuestra y qué no demuestra el experimento

La tabla solo muestra los primeros valores. La biyección se justifica demostrando que cada entero aparece exactamente una vez; observar muchos casos sirve para comprender el patrón, no para sustituir la prueba.

Aplicación integradora: álgebra de Boole y circuitos

La secuencia práctica adapta el capítulo 3 de Tocci y Widmer (2003): variables booleanas, tablas de verdad, compuertas OR, AND y NOT, traducción entre circuitos y expresiones, evaluación de salidas, NAND/NOR, De Morgan, universalidad y diagramas de temporización.

Compuertas básicas en R

```
NOT <- function(a) !a
OR  <- function(a, b) a | b
AND <- function(a, b) a & b
NOR <- function(a, b) !(a | b)
NAND <- function(a, b) !(a & b)

entradas_basicas <- expand.grid(
  b = c(TRUE, FALSE),
  a = c(TRUE, FALSE)
)[, c("a", "b")]

data.frame(
  entradas_basicas,
  OR = OR(entradas_basicas$a, entradas_basicas$b),
  AND = AND(entradas_basicas$a, entradas_basicas$b),
  NOR = NOR(entradas_basicas$a, entradas_basicas$b),
  NAND = NAND(entradas_basicas$a, entradas_basicas$b)
)
```

De circuito a expresión y de expresión a circuito

Para analizar un diagrama:

1. se identifican entradas, salidas y polaridades activas;
2. se asigna una variable intermedia a la salida de cada compuerta;
3. se escribe la expresión de cada etapa de izquierda a derecha;
4. se sustituye hasta obtener la salida en función de las entradas;
5. se comprueba el resultado mediante una tabla de verdad.

Para implantar una expresión se realiza el proceso inverso. Por ejemplo,

$$X = (A + \overline{B})C$$

requiere invertir B , formar $A + \overline{B}$ con OR y combinar el resultado con C mediante AND.

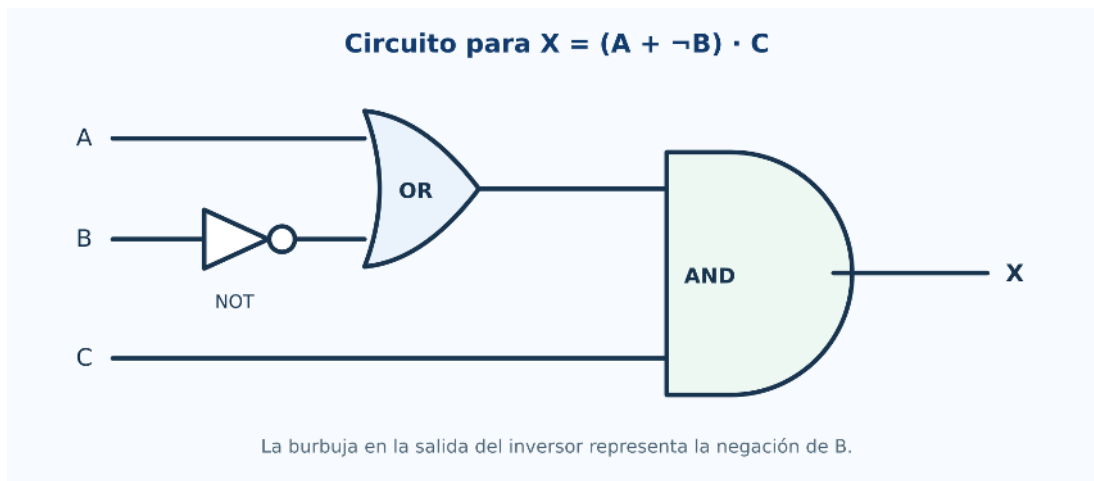


Figura 1.5: Implementación de $X = (A + \overline{B})C$ con inversor, compuerta OR y compuerta AND.

Comprobación de una simplificación

Estudiemos

$$F = \overline{a}b + ab + a\overline{b}.$$

Algebraicamente,

$$\begin{aligned}
 F &= b(\bar{a} + a) + a\bar{b} \\
 &= b + a\bar{b} \\
 &= (b + a)(b + \bar{b}) \\
 &= a + b.
 \end{aligned}$$

R comprueba la igualdad en las cuatro combinaciones:

```

tv <- expand.grid(
  b = c(TRUE, FALSE),
  a = c(TRUE, FALSE)
)[, c("a", "b")]

F_original <- (!tv$a & tv$b) |
              (tv$a & tv$b) |
              (tv$a & !tv$b)

F_simplificada <- tv$a | tv$b

data.frame(
  tv,
  F_original,
  F_simplificada,
  iguales = F_original == F_simplificada
)

all(F_original == F_simplificada)

```

Universalidad de NOR y NAND

```

nor <- function(x, y) !(x | y)

no_con_nor <- function(x) nor(x, x)
o_con_nor <- function(x, y) nor(nor(x, y), nor(x, y))
y_con_nor <- function(x, y) nor(nor(x, x), nor(y, y))

entradas <- expand.grid(
  b = c(TRUE, FALSE),

```

```

  a = c(TRUE, FALSE)
)[, c("a", "b")]

data.frame(
  entradas,
  NOT_a = no_con_nor(entradas$a),
  OR_ab = o_con_nor(entradas$a, entradas$b),
  AND_ab = y_con_nor(entradas$a, entradas$b)
)

```

La misma comprobación puede realizarse con NAND:

```

nand <- function(x, y) !(x & y)

no_con_nand <- function(x) nand(x, x)
y_con_nand <- function(x, y) nand(nand(x, y), nand(x, y))
o_con_nand <- function(x, y) nand(nand(x, x), nand(y, y))

data.frame(
  entradas,
  NOT_a = no_con_nand(entradas$a),
  OR_ab = o_con_nand(entradas$a, entradas$b),
  AND_ab = y_con_nand(entradas$a, entradas$b)
)

```

Señales activas en alto y en bajo

Una señal activa en alto realiza su función cuando vale 1; una activa en bajo la realiza cuando vale 0. En un nombre de señal, la barra superior puede indicar activación en bajo. En un símbolo lógico, una burbuja indica inversión o terminal activa en bajo.

```

habilitada_alta <- c(FALSE, TRUE, FALSE, TRUE)
habilitada_baja <- !habilitada_alta

data.frame(
  nivel_entrada = as.integer(habilitada_alta),
  activa_en_alto = habilitada_alta,
  activa_en_bajo = habilitada_baja
)

```

! Modelo lógico frente a circuito físico

En R, cero y uno se modelan como estados lógicos. En el dispositivo real corresponden a intervalos de voltaje definidos por su tecnología. El modelo de este capítulo no incluye todavía márgenes de ruido ni retardos de propagación.

Formas de onda lógicas

El siguiente ejemplo genera señales binarias y calcula las salidas AND y OR.

```
t <- 0:8
senal_A <- c(0, 0, 1, 1, 1, 0, 1, 1, 0)
senal_B <- c(1, 1, 1, 0, 0, 1, 0, 0, 0)

salida_AND <- as.integer(as.logical(senal_A) & as.logical(senal_B))
salida_OR <- as.integer(as.logical(senal_A) | as.logical(senal_B))

matplot(
  t,
  cbind(senal_A, senal_B, salida_AND, salida_OR),
  type = "s",
  lty = 1,
  lwd = 2,
  xlab = "Tiempo",
  ylab = "Nivel lógico",
  yaxt = "n",
  col = c("#1f77b4", "#d62728", "#2ca02c", "#9467bd")
)
axis(2, at = c(0, 1))
legend(
  "topright",
  legend = c("A", "B", "A AND B", "A OR B"),
  col = c("#1f77b4", "#d62728", "#2ca02c", "#9467bd"),
  lty = 1,
  lwd = 2,
  bty = "n"
)
```

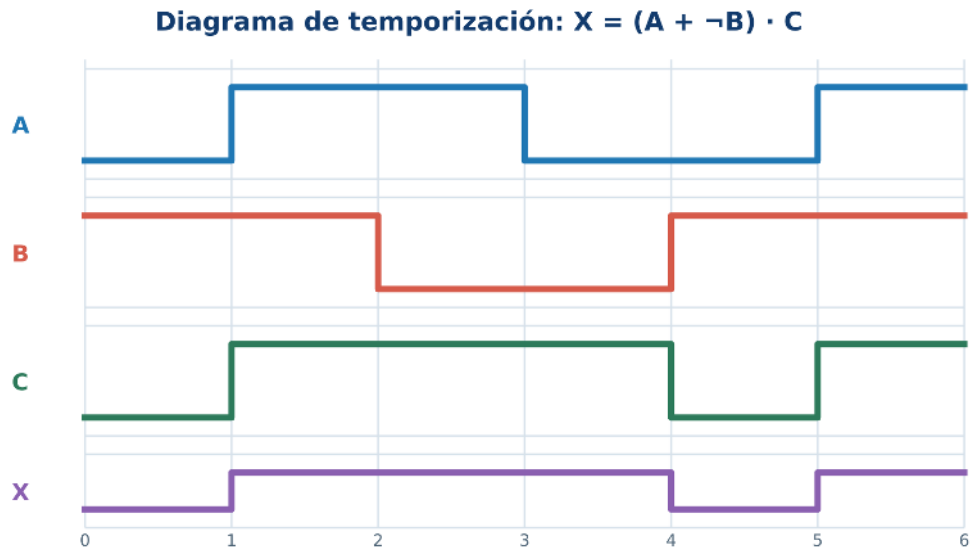


Figura 1.6: Las señales se dividen en intervalos constantes; en cada uno se evalúa $X = (A + \overline{B})C$.

i Lectura coordinada del circuito

Compare cada intervalo del diagrama de temporización con la fila correspondiente de la tabla de verdad y con el circuito de la Figura 1.5. Las compuertas y la burbuja del inversor siguen una convención gráfica uniforme basada en los símbolos lógicos estudiados por Tocci y Widmer (2003).

Laboratorio del capítulo

La edición integra tres aplicaciones que funcionan directamente en el sitio estático, sin servidor ni instalación adicional:

1. **Constructor de tablas de verdad:** selección de conectivos y clasificación como tautología, contradicción o contingencia.
2. **Operaciones de conjuntos:** regiones de Venn, cardinalidad e inclusión-exclusión.
3. **Clasificador de funciones:** diagramas sagitales y diagnóstico de inyectividad, suprayectividad y biyectividad.

El análisis de circuitos se apoya en figuras propias, tablas de verdad y código R reproducible. De esta manera se conserva la trazabilidad entre expresión, circuito y forma de onda sin depender de un servidor.

Ejercicios con R

1. Modifique U, A, B y compruebe ambas leyes de De Morgan.
2. Construya la tabla de verdad de $[(p \wedge \neg q) \Rightarrow (r \wedge \neg r)] \Leftrightarrow (p \Rightarrow q)$.
3. Escriba una función que determine si dos conjuntos finitos son disjuntos.
4. Extienda el programa de relaciones para indicar qué pares impiden la simetría o la transitividad.
5. Construya tres tablas: una función solo inyectiva, otra solo suprayectiva y una biyectiva.
6. Compruebe por tabla de verdad la ley de absorción $a + ab = a$.
7. Implemente NAND y demuestre computacionalmente que también es una compuerta universal.
8. Genere dos señales binarias propias y trace las salidas XOR y NOR.

Proyecto integrador

Diseñe un sistema lógico de tres entradas relacionado con sensores electrónicos:

1. describa la regla de activación;
2. escriba su expresión booleana;
3. genere la tabla de verdad en R;
4. simplifique la expresión manualmente;
5. compruebe la simplificación con R;
6. dibuje los circuitos original y simplificado;
7. genere una forma de onda de prueba;
8. interprete la reducción en el número de compuertas.

Recursos interactivos y multimedia

Esta versión ya incorpora:

- cuaderno completo del capítulo para Google Colab con R;
- laboratorio de tablas de verdad;

- laboratorio de operaciones con conjuntos y diagramas de Venn;
- clasificador de funciones mediante diagramas sagitales;
- figuras vectoriales para operaciones, relaciones, composición y tipos de funciones;
- circuito lógico propio para $X = (A + \bar{B})C$;
- diagrama de temporización coordinado con la expresión y la tabla de verdad.

Síntesis

R permite experimentar con conjuntos finitos, tablas de verdad, relaciones y funciones. Las comprobaciones exhaustivas son concluyentes cuando el dominio es finito y se han enumerado todos los casos. Para afirmaciones generales sobre conjuntos infinitos, la demostración matemática sigue siendo el fundamento.

Referencias del capítulo

La secuencia temática sigue el programa de Álgebra Superior (Universidad Autónoma de San Luis Potosí, Facultad de Ciencias 2011). Los fundamentos se apoyan en Silva y Lazo (2007), Epp (2011) y Rosen (2019). La aplicación a compuertas y álgebra booleana adapta el capítulo 3 de Tocci y Widmer (2003). El código utiliza R base (R Core Team 2026).

2 Números enteros y reales

Experimentación con R, algoritmos y recta numérica

Introducción

Este capítulo convierte propiedades de los enteros y los reales en experimentos reproducibles. R permitirá generar ejemplos, ejecutar algoritmos, detectar patrones y visualizar distancias. La comprobación computacional será una herramienta para formular y revisar conjeturas; las afirmaciones para infinitos números continuarán requiriendo demostración.

Todo el código principal usa R base (R Core Team 2026). Puede ejecutarse en RStudio o en Google Colab sin instalar paquetes.

* Cuaderno completo y ejecutable

El cuaderno sigue las secciones 2.1-2.8, incluye algoritmos de divisibilidad y primos, experimentos sobre precisión numérica y gráficas de exponentes y valor absoluto.

[Abrir en Google Colab](#) [Descargar el cuaderno](#)

Objetivos

Al finalizar, el lector podrá:

- representar enteros y operar con cociente y residuo en R;
- experimentar con sucesiones que se demuestran por inducción;
- programar los algoritmos de Euclides y Euclides extendido;
- detectar primos y obtener factorizaciones;
- distinguir cálculo exacto simbólico de aproximación numérica;
- explorar propiedades de orden y densidad en la recta real;
- evaluar potencias con exponentes enteros y racionales atendiendo a su dominio;
- resolver e interpretar inecuaciones con valor absoluto.

2.1 Propiedades de los números enteros

Enteros y almacenamiento en R

R distingue valores enteros escritos con el sufijo L y valores numéricos almacenados, normalmente, en doble precisión.

```
z <- c(-3L, -2L, -1L, 0L, 1L, 2L, 3L)

typeof(z)
typeof(3)
typeof(3L)
```

Para los ejemplos habituales ambos tipos producen las mismas operaciones. Sin embargo, una computadora representa un conjunto **finito** de valores; no contiene literalmente todos los elementos de $\mathbb{Z}$ ni de $\mathbb{R}$.

```
.Machine$integer.max
.Machine$double.xmax
```

! Modelo matemático y representación computacional

Los enteros matemáticos no tienen cota superior. Los enteros almacenados en un tipo de computadora sí la tienen. Cuando un cálculo excede el rango o la precisión disponible, el resultado puede desbordarse o redondearse.

Comprobación de propiedades

Elegimos tres enteros y comparamos ambos lados de las propiedades asociativa y distributiva.

```
a <- -7L
b <- 5L
c <- 3L

data.frame(
  propiedad = c("asociativa de la suma", "distributiva"),
  izquierda = c((a + b) + c, a * (b + c)),
  derecha = c(a + (b + c), a * b + a * c)
)
```

La igualdad para estos valores es una **verificación**, no una demostración para todos los enteros.

Orden en la recta numérica

```
x <- -6:6
plot(x, rep(0, length(x)), pch = 19,
     xlab = "entero", ylab = "", yaxt = "n",
     main = "Enteros en la recta numérica")
abline(h = 0, col = "gray50")
text(x, rep(0.08, length(x)), labels = x, cex = 0.8)
```

En la recta, $a < b$ significa que a aparece a la izquierda de b . Sumar el mismo número traslada ambos puntos sin cambiar su orden.

2.2 Inducción matemática

Explorar antes de demostrar

Consideremos la conjetura

$$1 + 3 + \cdots + (2n - 1) = n^2.$$

R permite comparar ambos lados para los primeros valores.

```
n <- 1:12
lado_izquierdo <- cumsum(2 * n - 1)
lado_derecho <- n^2

data.frame(
  n = n,
  suma_impares = lado_izquierdo,
  cuadrado = lado_derecho,
  diferencia = lado_izquierdo - lado_derecho
)
```

Que la columna **diferencia** sea cero sugiere la identidad, pero la demostración inductiva explica por qué se conserva al pasar de n a $n + 1$.

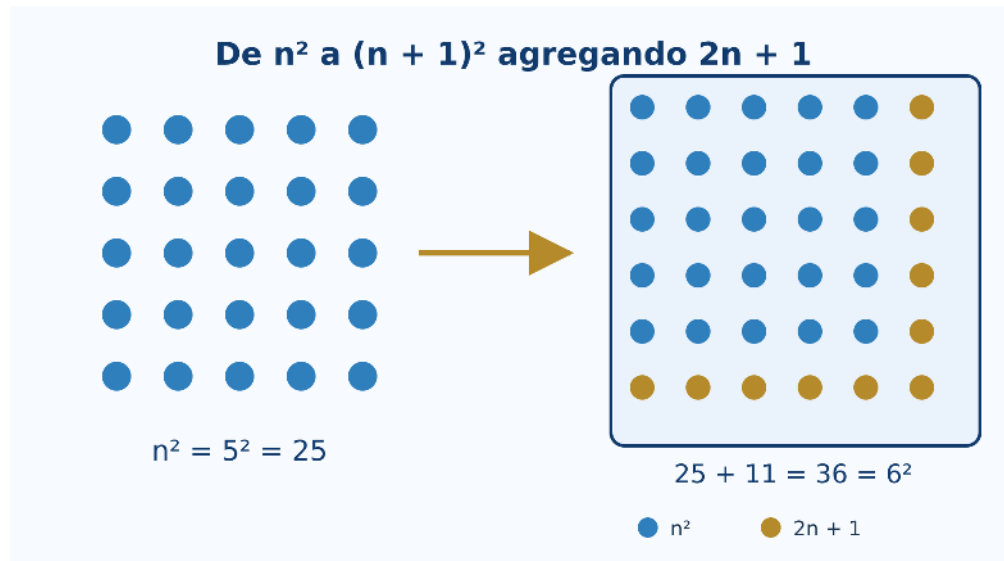


Figura 2.1: La suma acumulada de impares coincide con los cuadrados perfectos.

Función para revisar una identidad

```
revisar_identidad <- function(n_max = 50L) {
  n <- seq_len(n_max)
  izquierda <- cumsum(2 * n - 1)
  data.frame(
    n = n,
    izquierda = izquierda,
    derecha = n^2,
    coincide = izquierda == n^2
  )
}

resultado <- revisar_identidad(20L)
all(resultado$coincide)
```

Interactivo: construir el siguiente cuadrado

Seleccione n y observe cómo n^2 puntos, más una franja de $2n + 1$ puntos, forman $(n + 1)^2$.

[Abrir el laboratorio de inducción](#)

Lo que R no demuestra

No existe una instrucción que enumere todos los naturales y termine. La prueba formal requiere:

1. comprobar el caso base;
2. suponer la fórmula para un entero arbitrario k ;
3. deducir la fórmula para $k + 1$.

R ayuda a encontrar errores y contraejemplos. Si una afirmación falla para algún valor ensayado, queda refutada; si no falla, todavía necesita demostración.

2.3 Divisibilidad

Cociente y residuo

En R, `%/%` calcula el cociente entero y `%%` el residuo.

```
a <- c(47L, -47L)
d <- 6L
q <- a %/% d
r <- a %% d

data.frame(
  a = a,
  d = d,
  q = q,
  r = r,
  reconstruccion = d * q + r
)
```

Observe que R elige el residuo de modo que $0 \leq r < d$ cuando $d > 0$.

Una función de divisibilidad puede escribirse así:

```

divide <- function(a, b) {
  if (a == 0) stop("El divisor no puede ser cero")
  b %% a == 0
}

c(`7 divide 42` = divide(7L, 42L),
  `7 divide 40` = divide(7L, 40L))

```

Algoritmo de Euclides

```

euclides <- function(a, b) {
  a <- abs(as.integer(a))
  b <- abs(as.integer(b))
  pasos <- data.frame(
    dividendo = integer(), divisor = integer(),
    cociente = integer(), residuo = integer()
  )

  while (b != 0L) {
    q <- a %/% b
    r <- a %% b
    pasos <- rbind(
      pasos,
      data.frame(dividendo = a, divisor = b,
                  cociente = q, residuo = r)
    )
    a <- b
    b <- r
  }

  list(mcd = a, pasos = pasos)
}

resultado <- euclides(252L, 198L)
resultado$pasos
resultado$mcd

```

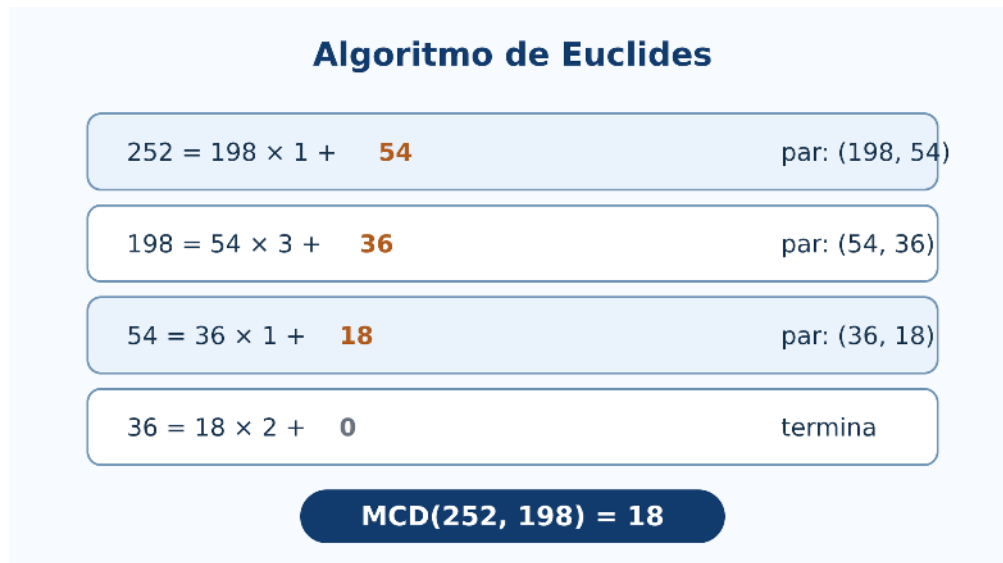



Figura 2.2: Los pares sucesivos del algoritmo de Euclides terminan en el máximo común divisor.

Euclides extendido y coeficientes de Bézout

```
euclides_extendido <- function(a, b) {  
  viejo_r <- as.integer(a); r <- as.integer(b)  
  viejo_s <- 1L; s <- 0L  
  viejo_t <- 0L; t <- 1L  
  
  while (r != 0L) {  
    q <- viejo_r %/% r  
    aux <- viejo_r - q * r; viejo_r <- r; r <- aux  
    aux <- viejo_s - q * s; viejo_s <- s; s <- aux  
    aux <- viejo_t - q * t; viejo_t <- t; t <- aux  
  }  
  
  if (viejo_r < 0L) {  
    viejo_r <- -viejo_r  
    viejo_s <- -viejo_s  
    viejo_t <- -viejo_t  
  }  
  c(mcd = viejo_r, x = viejo_s, y = viejo_t)  
}
```

```

}

bezout <- euclides_extendido(252L, 198L)
bezout
252L * bezout["x"] + 198L * bezout["y"]

```

Interactivo: residuos sucesivos

Introduzca dos enteros positivos. El laboratorio muestra cada división y señala el último residuo no nulo.

[Abrir el algoritmo de Euclides](#)

2.4 Números primos y factorización

Prueba de primalidad por división

Para decidir si n es primo basta probar divisores hasta $\sqrt{n}$.

```

es_primo <- function(n) {
  n <- as.integer(n)
  if (n < 2L) return(FALSE)
  if (n == 2L) return(TRUE)
  if (n %% 2L == 0L) return(FALSE)

  limite <- floor(sqrt(n))
  if (limite < 3L) return(TRUE)
  candidatos <- seq.int(3L, limite, by = 2L)
  !any(n %% candidatos == 0L)
}

sapply(1:30, es_primo)
(1:100)[sapply(1:100, es_primo)]

```

Criba de Eratóstenes

La criba elimina múltiplos y conserva los primos no mayores que un límite.

```
criba <- function(n) {  
  n <- as.integer(n)  
  if (n < 2L) return(integer())  
  primo <- rep(TRUE, n)  
  primo[1] <- FALSE  
  
  limite <- floor(sqrt(n))  
  if (limite >= 2L) {  
    for (p in 2:limite) {  
      if (primo[p]) primo[seq.int(p * p, n, by = p)] <- FALSE  
    }  
  }  
  which(primo)  
}  
  
criba(100L)
```

Factorización por divisiones sucesivas

```
factorizar <- function(n) {  
  n <- abs(as.integer(n))  
  if (n < 2L) return(integer())  
  factores <- integer()  
  d <- 2L  
  
  while (d * d <= n) {  
    while (n %% d == 0L) {  
      factores <- c(factores, d)  
      n <- n %/% d  
    }  
    d <- if (d == 2L) 3L else d + 2L  
  }  
  if (n > 1L) factores <- c(factores, n)  
  factores  
}
```

```
factorizar(756L)
table(factorizar(756L))
```

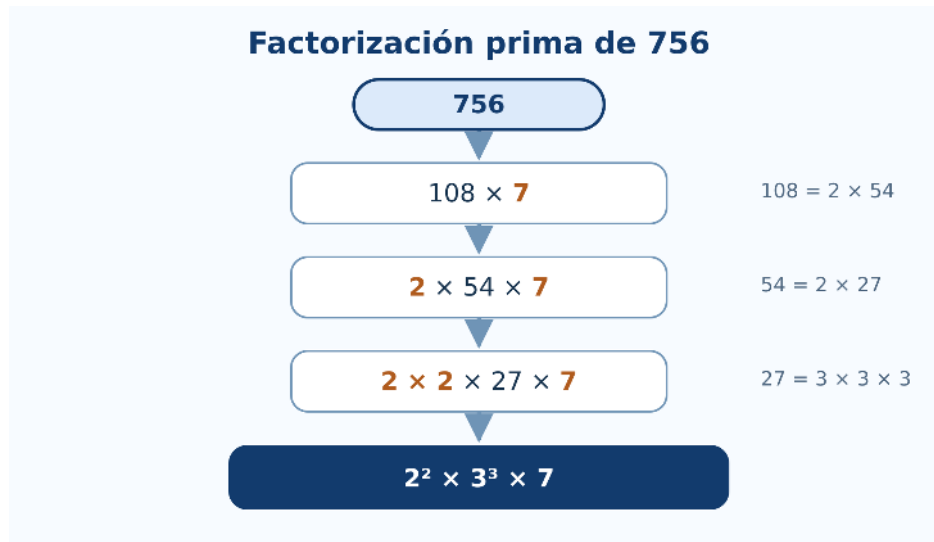


Figura 2.3: Cadena de factorización y descomposición prima de 756.

MCD y MCM

```
mcd <- function(a, b) euclides(a, b)$mcd
mcm <- function(a, b) {
  if (a == 0L || b == 0L) return(0L)
  abs(a %/% mcd(a, b) * b)
}

a <- 840L
b <- 378L
c(mcd = mcd(a, b), mcm = mcm(a, b), producto = a * b)
mcd(a, b) * mcm(a, b) == a * b
```

2.5 Números racionales y números reales

Representar una fracción exactamente

R base representa $1/3$ mediante una aproximación de punto flotante. Para conservar una fracción exacta podemos guardar numerador y denominador reducidos.

```
fraccion <- function(num, den = 1L) {  
  if (den == 0L) stop("El denominador no puede ser cero")  
  num <- as.integer(num)  
  den <- as.integer(den)  
  if (den < 0L) { num <- -num; den <- -den }  
  g <- mcd(abs(num), den)  
  c(num = num %/% g, den = den %/% g)  
}  
  
fraccion(18L, 24L)  
fraccion(-15L, -35L)
```

Aproximación y precisión

El conocido ejemplo siguiente muestra que una igualdad matemáticamente correcta puede no ser exacta en aritmética binaria de punto flotante.

```
0.1 + 0.2  
(0.1 + 0.2) == 0.3  
all.equal(0.1 + 0.2, 0.3)
```

Para comparar resultados numéricos se utiliza una tolerancia:

```
iguales_aprox <- function(x, y, tolerancia = 1e-12) {  
  abs(x - y) <= tolerancia  
}  
  
iguales_aprox(0.1 + 0.2, 0.3)
```

Racionales e irracionales en la recta

```
valores <- c(0, 1/2, sqrt(2), 3/2, sqrt(3), 2)
etiquetas <- c("0", "1/2", "sqrt(2)", "3/2", "sqrt(3)", "2")

plot(valores, rep(0, length(valores)), pch = 19,
      xlim = c(-0.1, 2.1), ylim = c(-0.35, 0.35),
      axes = FALSE, xlab = "recta real", ylab = "")
axis(1)
abline(h = 0, col = "gray50")
text(valores, rep(0.12, length(valores)), etiquetas, cex = 0.8)
```

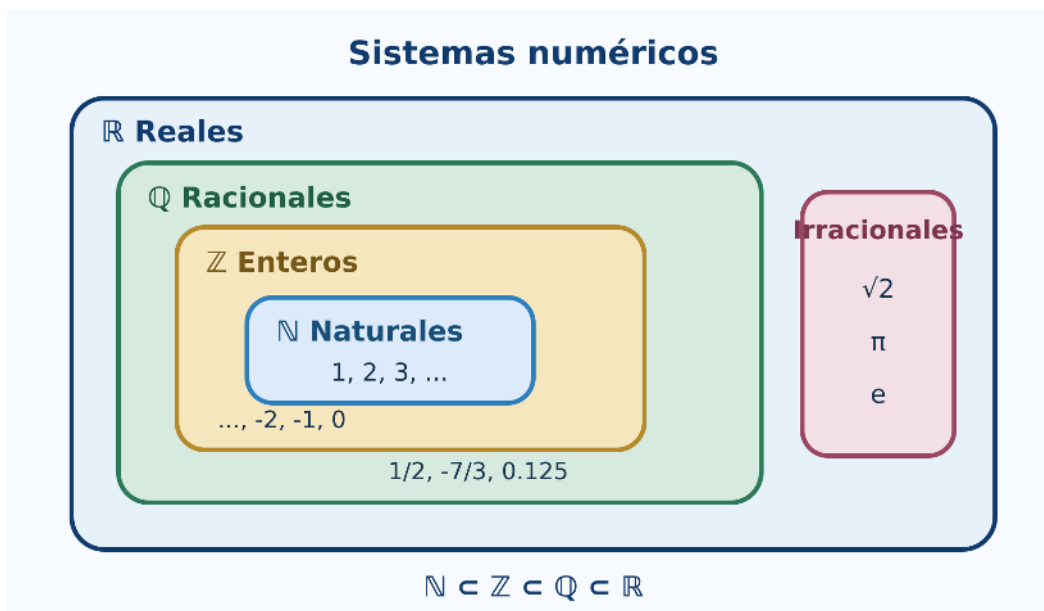


Figura 2.4: Los naturales, enteros, racionales e irracionales se sitúan dentro de los reales.

2.6 Propiedades de los números reales

Orden y operaciones

Podemos observar que multiplicar por un número negativo invierte una desigualdad.

```

a <- -2
b <- 5
c_positivo <- 3
c_negativo <- -3

data.frame(
  comparacion = c("original", "multiplicar por positivo", "multiplicar por negativo"),
  izquierda = c(a, a * c_positivo, a * c_negativo),
  derecha = c(b, b * c_positivo, b * c_negativo),
  menor = c(a < b,
             a * c_positivo < b * c_positivo,
             a * c_negativo < b * c_negativo)
)

```

La tercera fila es falsa porque el sentido correcto es $ac > bc$ cuando $c < 0$.

Densidad mediante puntos intermedios

Entre $a < b$ podemos construir repetidamente puntos medios.

```

puntos_medios <- function(a, b, niveles = 4L) {
  puntos <- c(a, b)
  for (i in seq_len(niveles)) {
    puntos <- sort(unique(c(puntos, head(puntos, -1) + diff(puntos) / 2)))
  }
  puntos
}

p <- puntos_medios(1, 2, 4L)
p
plot(p, rep(0, length(p)), pch = 19,
     xlab = "x", ylab = "", yaxt = "n",
     main = "Puntos intermedios entre 1 y 2")
abline(h = 0, col = "gray50")

```

Aproximar $\sqrt{2}$ por bisección

La completitud garantiza en $\mathbb{R}$ la existencia del límite que llena el corte entre números con cuadrado menor y mayor que 2.

```

aproximar_raiz2 <- function(iteraciones = 20L) {
  inferior <- 1
  superior <- 2
  historial <- data.frame()

  for (i in seq_len(iteraciones)) {
    medio <- (inferior + superior) / 2
    historial <- rbind(
      historial,
      data.frame(i = i, inferior = inferior,
                  medio = medio, superior = superior,
                  error = medio^2 - 2)
    )
    if (medio^2 < 2) inferior <- medio else superior <- medio
  }
  historial
}

aproximacion <- aproximar_raiz2(15L)
tail(aproximacion, 5)
sqrt(2)

```

2.7 Exponentes racionales y negativos

Potencias y dominio

```

data.frame(
  expresion = c("2^(-3)", "16^(3/4)", "(-8)^(1/3)"),
  valor_R = c(2^(-3), 16^(3/4), (-8)^(1/3))
)

```

La última expresión muestra una dificultad: aunque la raíz cúbica real de -8 es -2 , la operación directa puede producir NaN porque R evalúa potencias reales mediante logaritmos complejos fuera del dominio real positivo.

Podemos construir la raíz impar real:


```

raiz_impar <- function(x, n) {
  if (n <= 0L || n %% 2L == 0L) stop("n debe ser impar positivo")
  sign(x) * abs(x)^(1 / n)
}

raiz_impar(-8, 3L)

```

Verificar leyes con restricciones

```

a <- 5
r <- 2/3
s <- -1/4

c(
  producto = a^r * a^s,
  suma_exponentes = a^(r + s),
  diferencia = a^r * a^s - a^(r + s)
)

```

Por redondeo, conviene comparar con `all.equal()`.

```
all.equal(a^r * a^s, a^(r + s))
```

Gráficas de potencias

```

x <- seq(0.05, 4, length.out = 400)
plot(x, x^2, type = "l", lwd = 2,
     xlab = "x", ylab = "y", ylim = c(0, 8),
     main = "Exponentes sobre bases positivas")
lines(x, sqrt(x), lwd = 2, lty = 2)
lines(x, x^(-1), lwd = 2, lty = 3)
legend("topright", c("x^2", "x^(1/2)", "x^(-1)"),
     lty = c(1, 2, 3), lwd = 2, bty = "n")

```

2.8 Valor absoluto

Distancia en la recta

```
x <- c(-5, -2, 0, 3, 7)
a <- 2
data.frame(x = x, centro = a, distancia = abs(x - a))
```

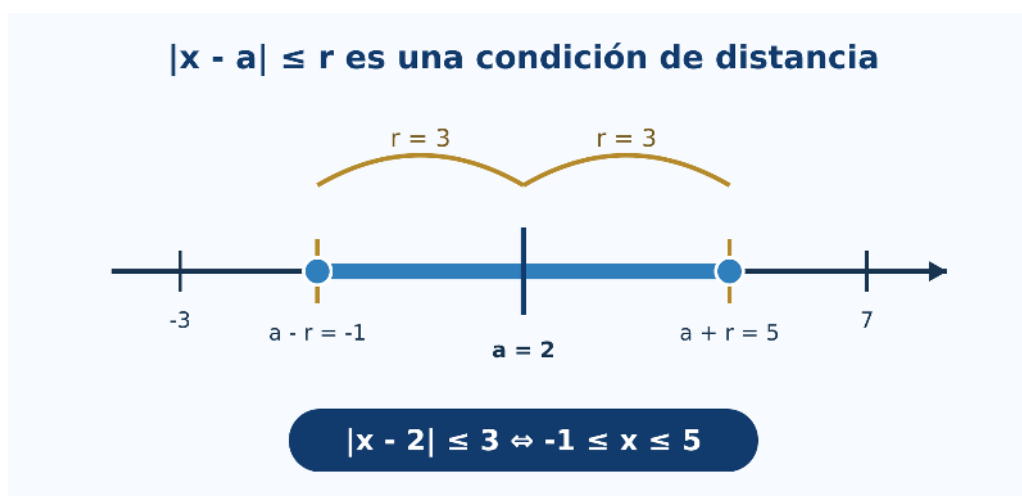


Figura 2.5: La desigualdad $|x - a| \leq r$ selecciona un intervalo centrado en a .

Resolver inecuaciones

Para resolver $|x - a| \leq r$, el intervalo solución es $[a - r, a + r]$ cuando $r \geq 0$.

```
intervalo_abs <- function(a, r, tipo = c("menor_igual", "mayor")) {
  tipo <- match.arg(tipo)
  if (r < 0) stop("r debe ser no negativo")
  extremos <- c(a - r, a + r)
  if (tipo == "menor_igual") {
    list(intervalo = extremos,
         descripcion = sprintf("[%g, %g]", extremos[1], extremos[2]))
  } else {
    list(intervalos = matrix(c(-Inf, extremos[1], extremos[2], Inf), nrow = 2, byrow = TRUE
```

```

        descripcion = sprintf("(-Inf, %g) U (%g, Inf)", extremos[1], extremos[2]))
    }
}

intervalo_abs(a = 4, r = 7, tipo = "menor_igual")
intervalo_abs(a = -1, r = 2, tipo = "mayor")

```

Comprobar la desigualdad triangular

```

set.seed(2026)
x <- runif(1000, -20, 20)
y <- runif(1000, -20, 20)

all(abs(x + y) <= abs(x) + abs(y) + 1e-12)
max(abs(x + y) - (abs(x) + abs(y)))

```

La simulación busca contraejemplos en mil pares. No sustituye la demostración, pero ayuda a comprobar la implementación y a observar que la diferencia nunca resulta positiva.

Interactivo: centro, radio e intervalo

Cambie el centro a , el radio r y el tipo de desigualdad. La recta numérica muestra si la solución es un intervalo central o dos regiones exteriores.

[Abrir el laboratorio de valor absoluto](#)

Laboratorio del capítulo

La versión incorpora tres actividades autónomas:

1. **Inducción visual:** construye $(n + 1)^2$ a partir de n^2 y $2n + 1$.
2. **Algoritmo de Euclides:** presenta cocientes y residuos hasta encontrar el MCD.
3. **Valor absoluto:** transforma distancia en intervalos y regiones de la recta real.

Estas actividades funcionan en el sitio estático. El cuaderno Colab reúne además todos los algoritmos y experimentos de R.

Ejercicios con R

1. Genere enteros aleatorios y compruebe las propiedades conmutativa, asociativa y distributiva.
2. Construya una tabla para contrastar $1 + 2 + \dots + n$ con $n(n + 1)/2$ hasta $n = 100$.
3. Modifique `euclides()` para aceptar una lista de enteros y calcular su MCD.
4. Use `euclides_extendido()` para resolver una ecuación diofántica $ax + by = c$.
5. Compare el tiempo de `es_primo()` y `criba()` al buscar primos hasta un límite creciente.
6. Escriba una función que devuelva la factorización como texto, por ejemplo $2^2 * 3^3 * 7$.
7. Genere aproximaciones racionales de $\sqrt{2}$ y grafique el error absoluto.
8. Construya ejemplos donde comparar con `==` falle, pero `all.equal()` sea verdadero.
9. Trace x^r para exponentes negativos, fraccionarios y enteros en sus dominios reales.
10. Genere puntos al azar y compruebe la desigualdad triangular inversa.

Proyecto integrador

Desarrolle un pequeño analizador de enteros que reciba a y b y produzca:

1. clasificación de signo y paridad;
2. divisores positivos;
3. factorización prima;
4. tabla completa del algoritmo de Euclides;
5. coeficientes de Bézout;
6. MCD y MCM calculados de dos maneras;
7. verificaciones automáticas de las identidades utilizadas;
8. una interpretación escrita de los resultados.

El informe debe distinguir con claridad qué resultados son cálculos, cuáles son verificaciones finitas y cuáles dependen de un teorema general.

Síntesis

R hace visibles los algoritmos y permite experimentar con numerosos ejemplos. El residuo implementa divisibilidad; Euclides reduce el problema del MCD; la criba y las divisiones sucesivas exploran primos; las aproximaciones muestran la diferencia entre un

real matemático y su representación finita. La inducción, la unicidad de la factorización y las propiedades de completitud siguen requiriendo razonamiento matemático.

Referencias del capítulo

La secuencia sigue la unidad 2 del programa de Álgebra Superior (Universidad Autónoma de San Luis Potosí, Facultad de Ciencias 2011). Los fundamentos se apoyan en Silva y Lazo (2007), Rosen (2019), Burton (2011) y Niven et al. (1991). Los algoritmos se implementan con R base (R Core Team 2026).

3 Números complejos

Introducción

Los números complejos unen álgebra y geometría. En forma cartesiana se operan mediante sus partes real e imaginaria; en forma polar, la multiplicación se convierte en una combinación de cambio de escala y rotación. R incorpora números complejos en su lenguaje base, por lo que permite experimentar sin instalar paquetes.

El capítulo sigue las nueve secciones de la Unidad 3 del programa oficial de Álgebra Superior de la Facultad de Ciencias de la UASLP (Universidad Autónoma de San Luis Potosí, Facultad de Ciencias 2011). Cada sección comienza con el procedimiento manual, lo traduce a R y termina con una interpretación geométrica.

Objetivos

Al terminar, el estudiante podrá:

- representar pares, vectores y complejos en el plano;
- convertir coordenadas cartesianas y polares;
- programar operaciones vectoriales con R base;
- utilizar `Re()`, `Im()`, `Mod()`, `Arg()` y `Conj()`;
- interpretar sumas, productos, conjugados y cocientes;
- calcular potencias y todas las raíces de un complejo;
- comprobar resultados numéricamente sin confundir una prueba experimental con una demostración.

3.1 Definición de $\mathbb{R}^2$

Un elemento de $\mathbb{R}^2$ es un par ordenado (x, y) . Puede representar un punto o el vector dirigido desde el origen hasta ese punto.

Situación inicial

Un robot se desplaza 3 unidades al este y 4 al norte. Su posición final es (3, 4) y la distancia al origen es

$$\sqrt{3^2 + 4^2} = 5.$$

En R, un vector de dos componentes puede guardarse como un vector numérico.

```
p <- c(x = 3, y = 4)
p
sqrt(sum(p^2))
```

Para varios puntos conviene usar un data.frame.

```
puntos <- data.frame(
  nombre = c("A", "B", "C", "D"),
  x = c(3, -2, -3, 2),
  y = c(4, 3, -2, -3)
)

puntos$cuadrante <- with(puntos, ifelse(
  x > 0 & y > 0, "I",
  ifelse(x < 0 & y > 0, "II",
    ifelse(x < 0 & y < 0, "III", "IV"))
))
puntos
```

Plano cartesiano en R

```
plot(puntos$x, puntos$y,
     xlim = c(-5, 5), ylim = c(-5, 5), asp = 1,
     xlab = "x", ylab = "y", pch = 19, col = "#1769a6",
     main = "Puntos de R2")
abline(h = 0, v = 0, col = "gray65")
text(puntos$x, puntos$y, labels = puntos$nombre,
     pos = 3, col = "#123b6d")
```

Distancia entre dos puntos

```
distancia <- function(P, Q) sqrt(sum((P - Q)^2))  
  
P <- c(-2, 3)  
Q <- c(4, -1)  
distancia(P, Q)
```

i Interpretación

El código reproduce la definición. $P - Q$ obtiene las diferencias de componentes, 2 calcula sus cuadrados, $\text{sum}()$ los suma y $\text{sqrt}()$ extrae la raíz.

3.2 Representaciones cartesiana y polar de vectores en $\mathbb{R}^2$

Para $(x, y) \neq (0, 0)$:

$$r = \sqrt{x^2 + y^2}, \quad \theta = \text{atan2}(y, x).$$

La función de dos argumentos $\text{atan2}(y, x)$ identifica el cuadrante correcto. La función $\text{atan}(y/x)$ no lo hace en todos los casos.

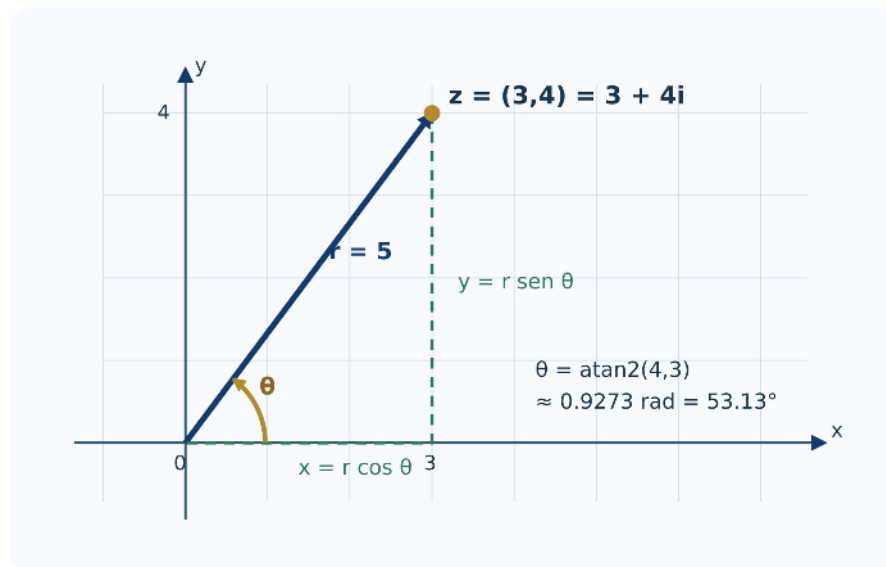


Figura 3.1: Del par cartesiano $(3, 4)$ a radio 5 y argumento θ .

Conversión cartesiana a polar

```
cartesiana_polar <- function(x, y) {
  r <- sqrt(x^2 + y^2)
  theta <- ifelse(r == 0, NA_real_, atan2(y, x))
  data.frame(x = x, y = y, r = r,
             theta_rad = theta,
             theta_grados = theta * 180 / pi)
}

cartesiana_polar(3, 4)
cartesiana_polar(-sqrt(3), 1)
cartesiana_polar(0, 0)
```

Conversión polar a cartesiana

```
polar_cartesiana <- function(r, theta) {
  if (any(r < 0)) stop("El radio debe ser no negativo")
```

```

data.frame(r = r, theta = theta,
           x = r * cos(theta),
           y = r * sin(theta))
}

polar_cartesiana(2, 5 * pi / 6)

```

Construcción equivalente en GeoGebra

En la vista gráfica pueden escribirse las órdenes:

```

O = (0, 0)
Z = (3, 4)
u = Vector(O, Z)
r = Length(u)
theta = Angle((1, 0), u)

```

Al arrastrar Z , GeoGebra actualiza simultáneamente sus coordenadas, módulo y argumento.

3.3 Operaciones con vectores en $\mathbb{R}^2$

Sean $\mathbf{u} = (u_1, u_2)$ y $\mathbf{v} = (v_1, v_2)$. La suma, resta, multiplicación por escalar y producto punto se calculan componente a componente.

```

u <- c(2, 1)
v <- c(-1, 2)

list(
  suma = u + v,
  resta = u - v,
  triple_u = 3 * u,
  producto_punto = sum(u * v)
)

```

Como $\text{sum}(\mathbf{u} * \mathbf{v})$ vale cero, los vectores son perpendiculares.

Regla del paralelogramo

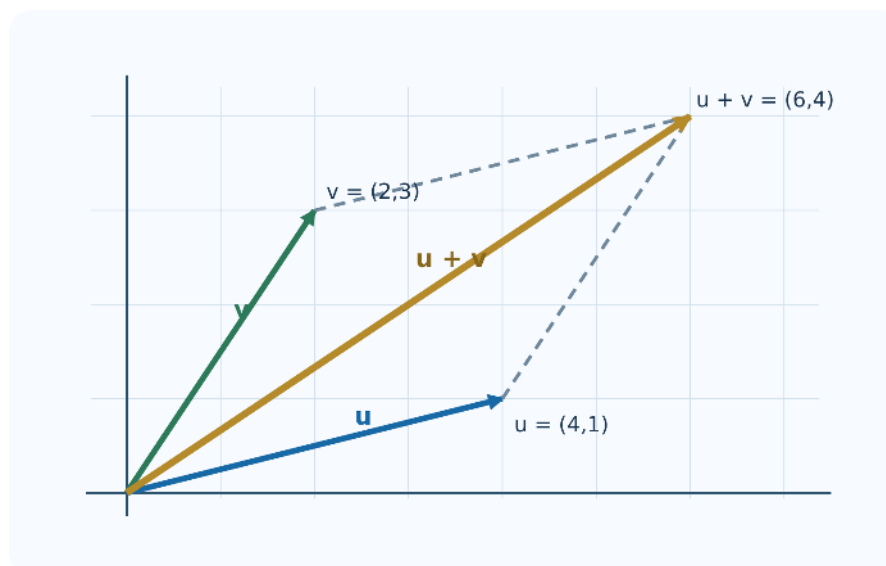


Figura 3.2: La diagonal representa $\mathbf{u} + \mathbf{v}$.

```
dibujar_suma <- function(u, v) {  
  s <- u + v  
  limites_x <- range(c(0, u[1], v[1], s[1])) + c(-1, 1)  
  limites_y <- range(c(0, u[2], v[2], s[2])) + c(-1, 1)  
  plot(0, 0, type = "n", xlim = limites_x, ylim = limites_y,  
       asp = 1, xlab = "x", ylab = "y",  
       main = "Suma vectorial")  
  abline(h = 0, v = 0, col = "gray75")  
  arrows(0, 0, u[1], u[2], length = 0.09,  
        col = "#1769a6", lwd = 3)  
  arrows(0, 0, v[1], v[2], length = 0.09,  
        col = "#2c7a5a", lwd = 3)  
  arrows(0, 0, s[1], s[2], length = 0.09,  
        col = "#b58a2a", lwd = 3)  
  segments(u[1], u[2], s[1], s[2], lty = 2, col = "gray45")  
  segments(v[1], v[2], s[1], s[2], lty = 2, col = "gray45")  
}  
  
dibujar_suma(c(3, 1), c(1, 2))
```

Verificación de propiedades

Los cálculos con punto flotante deben compararse con tolerancia.

```
iguales_vectores <- function(a, b, tol = 1e-12) {  
  length(a) == length(b) && all(abs(a - b) < tol)  
}  
  
w <- c(-2, 4)  
lambda <- 2.5  
  
c(  
  conmutativa = iguales_vectores(u + v, v + u),  
  asociativa = iguales_vectores((u + v) + w, u + (v + w)),  
  distributiva = iguales_vectores(lambda * (u + v),  
                                   lambda * u + lambda * v)  
)
```

3.4 Módulo y argumento

En el plano complejo se escribe

$$|z| = \sqrt{x^2 + y^2}, \quad \arg(z) = \theta.$$

R calcula estas cantidades con `Mod()` y `Arg()`.

```
z <- 3 + 4i  
c(modulo = Mod(z),  
  argumento_rad = Arg(z),  
  argumento_grados = Arg(z) * 180 / pi)
```

Argumento principal y ángulos equivalentes

```
z_cuadrantes <- c(1 + 1i, -1 + 1i, -1 - 1i, 1 - 1i)  
data.frame(  
  z = z_cuadrantes,
```

```
argumento_rad = Arg(z_cuadrantes),
argumento_grados = Arg(z_cuadrantes) * 180 / pi
)
```

`Arg()` devuelve el argumento principal en el intervalo $(-\pi, \pi]$. A cada resultado se le puede sumar $2k\pi$ sin cambiar el punto.

! El argumento de cero

R devuelve `Arg(0) = 0` por conveniencia computacional. Matemáticamente, el argumento de 0 no está definido porque el vector nulo no señala ninguna dirección. La función `cartesiana_polar()` registra este caso como NA.

Desigualdad triangular: experimento reproducible

```
set.seed(314)
z <- complex(real = runif(1000, -5, 5),
             imaginary = runif(1000, -5, 5))
w <- complex(real = runif(1000, -5, 5),
             imaginary = runif(1000, -5, 5))

holgura <- Mod(z) + Mod(w) - Mod(z + w)
summary(holgura)
all(holgura >= -1e-12)
```

! Alcance de la comprobación

Mil casos correctos apoyan la propiedad y ayudan a detectar errores, pero no demuestran que sea verdadera para todos los complejos. La demostración general aparece en el volumen de Fundamentos Matemáticos.

3.5 Números imaginarios y complejos

R reconoce un literal complejo cuando aparece el sufijo `i`. Debe escribirse `1i`, no solamente `i`, a menos que antes se haya creado un objeto con ese nombre.

```

z <- 3 + 4i
typeof(z)

c(
  parte_real = Re(z),
  parte_imaginaria = Im(z),
  modulo = Mod(z),
  argumento = Arg(z)
)

```

Construcción desde componentes

```

z1 <- 3 + 4i
z2 <- complex(real = 3, imaginary = 4)
z3 <- as.complex(3) + 4i

c(z1 = z1, z2 = z2, z3 = z3)
identical(z1, z2)

```

Vector de números complejos

```

zs <- c(2 + 1i, -1 + 3i, -2 - 2i, 3 - 1i)
tabla_z <- data.frame(
  z = zs,
  real = Re(zs),
  imaginaria = Im(zs),
  modulo = Mod(zs),
  argumento = Arg(zs)
)
tabla_z

```

Plano complejo en R

```
plot(Re(zs), Im(zs), asp = 1, pch = 19,
     xlab = "Parte real", ylab = "Parte imaginaria",
     col = "#1769a6", main = "Números complejos")
abline(h = 0, v = 0, col = "gray65")
text(Re(zs), Im(zs), labels = paste0("z", seq_along(zs)), pos = 3)
```

3.6 Operaciones básicas con números complejos

Solución manual

Para $z = 3 - 2i$ y $w = 1 + 4i$:

$$z + w = 4 + 2i,$$

$$zw = (3)(1) - (-2)(4) + ((3)(4) + (-2)(1))i = 11 + 10i.$$

Comprobación en R

```
z <- 3 - 2i
w <- 1 + 4i

c(suma = z + w,
  resta = z - w,
  producto = z * w)
```

Multiplicar significa rotar y escalar

```
resumen_producto <- function(z, w) {
  producto <- z * w
  data.frame(
    cantidad = c("z", "w", "z*w"),
    modulo = c(Mod(z), Mod(w), Mod(producto)),
    argumento = c(Arg(z), Arg(w), Arg(producto))
  )
}
```

```

)
}

resumen_producto(2 * exp(1i * pi / 6),
                 1.5 * exp(1i * pi / 3))

```

La tabla permite comprobar

$$|zw| = |z||w|, \quad \arg(zw) \equiv \arg(z) + \arg(w) \pmod{2\pi}.$$

3.7 Complejo conjugado y sus propiedades

El conjugado de $z = a + bi$ es $\bar{z} = a - bi$. R lo calcula con `Conj()`.

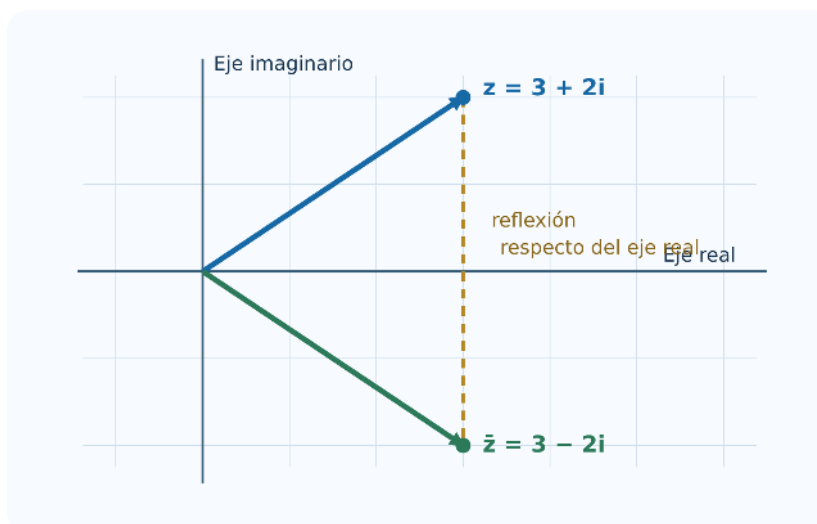


Figura 3.3: Conjugar refleja un punto respecto del eje real.

```

z <- 3 + 4i

c(
  z = z,
  conjugado = Conj(z),
  producto = z * Conj(z),

```



```
modulo_cuadrado = Mod(z)^2
)
```

Comprobaciones vectorizadas

```
z <- c(1 + 2i, -3 + 4i, 2 - 5i)
w <- c(2 - 1i, 1 + 3i, -4 + 2i)

c(
  involucion = all(Conj(Conj(z)) == z),
  suma = all(Conj(z + w) == Conj(z) + Conj(w)),
  producto = all(Conj(z * w) == Conj(z) * Conj(w)),
  modulo = all(abs(Mod(Conj(z)) - Mod(z)) < 1e-12)
)
```

Recuperar las partes real e imaginaria

```
z <- -2 + 7i
c(
  real_por_conjugado = (z + Conj(z)) / 2,
  imaginaria_por_conjugado = (z - Conj(z)) / (2i)
)
```

El segundo resultado es el número real 7, no $7i$, porque la fórmula calcula $\text{Im}(z)$.

3.8 División de complejos

Procedimiento manual

$$\frac{2+3i}{1-2i} = \frac{(2+3i)(1+2i)}{1^2 + (-2)^2} = -\frac{4}{5} + \frac{7}{5}i.$$

Cálculo y verificación en R

```
z <- 2 + 3i
w <- 1 - 2i
cociente <- z / w

c(cociente = cociente,
  comprobacion = cociente * w,
  error = Mod(cociente * w - z))
```

Programar la fórmula cartesiana

```
dividir_complejos <- function(z, w) {
  if (any(Mod(w) == 0)) stop("No se puede dividir entre cero")
  z * Conj(w) / Mod(w)^2
}

dividir_complejos(2 + 3i, 1 - 2i)
```

Comparación polar

```
resumen_cociente <- function(z, w) {
  q <- z / w
  data.frame(
    calculo = c("|z|/|w|", "|z/w|",
                "Arg(z)-Arg(w)", "Arg(z/w)"),
    valor = c(Mod(z) / Mod(w), Mod(q),
              Arg(z) - Arg(w), Arg(q))
  )
}

resumen_cociente(2 + 3i, 1 - 2i)
```

Los dos argumentos pueden diferir en un múltiplo de 2π y representar la misma dirección.

3.9 Potencias y raíces de complejos

Potencias con De Moivre

```
potencia_demoivre <- function(z, n) {  
  if (length(n) != 1L || !is.finite(n) || abs(n - round(n)) > sqrt(.Machine$double.eps)) {  
    stop("n debe ser un entero finito")  
  }  
  n <- as.integer(round(n))  
  if (z == 0 && n < 0) stop("Cero no admite exponentes negativos")  
  Mod(z)^n * exp(1i * n * Arg(z))  
}  
  
z <- 1 + 1i  
c(R = z^8,  
  De_Moivre = potencia_demoivre(z, 8),  
  diferencia = Mod(z^8 - potencia_demoivre(z, 8)))
```

R utiliza la identidad de Euler

$$e^{i\theta} = \cos \theta + i \sin \theta.$$

Por eso $r * \exp(1i * \theta)$ es una forma compacta de construir un complejo polar.

Todas las raíces n -ésimas

```
raices_n <- function(z, n) {  
  if (length(z) != 1L || length(n) != 1L || n < 1 || n != floor(n)) {  
    stop("Use un complejo y un entero positivo n")  
  }  
  if (z == 0) return(0 + 0i)  
  k <- 0:(n - 1)  
  Mod(z)^(1 / n) * exp(1i * (Arg(z) + 2 * pi * k) / n)  
}  
  
raices_cubicas <- raices_n(-8, 3)
```

```
raices_cubicas
raices_cubicas^3
```

Raíces de la unidad

```
raices_unidad <- function(n) exp(2 * pi * 1i * (0:(n - 1)) / n)

omega <- raices_unidad(6)
round(omega, 10)
```

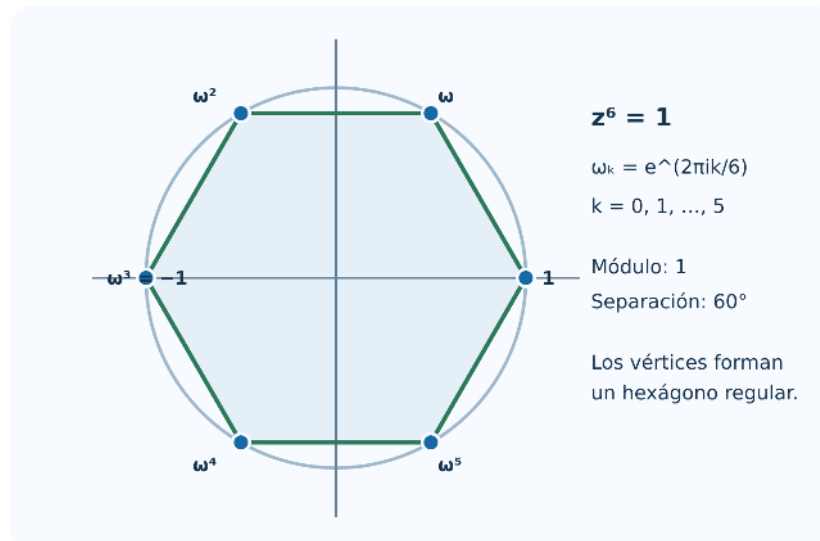


Figura 3.4: Las raíces sextas de la unidad son los vértices de un hexágono regular.

```
plot(Re(omega), Im(omega), asp = 1, pch = 19,
     xlim = c(-1.25, 1.25), ylim = c(-1.25, 1.25),
     xlab = "Parte real", ylab = "Parte imaginaria",
     col = "#1769a6", main = "Raíces sextas de la unidad")
abline(h = 0, v = 0, col = "gray75")
segments(Re(omega), Im(omega),
         Re(c(omega[-1], omega[1])), Im(c(omega[-1], omega[1])),
         col = "#2c7a5a", lwd = 2)
text(Re(omega), Im(omega), labels = paste0("k=", 0:5), pos = 3)
```

Laboratorio del capítulo

Trabaje en este orden:

1. elija un punto en el laboratorio cartesiano-polar y anote x , y , r y θ ;
2. compruebe manualmente $r = \sqrt{x^2 + y^2}$;
3. use el laboratorio de multiplicación para observar la suma de argumentos;
4. reproduzca el mismo ejemplo con R;
5. cambie n en el laboratorio de raíces y describa el polígono obtenido;
6. compruebe en R que cada punto elevado a n produce 1 dentro de la tolerancia numérica.

Ejercicios con R

1. Escriba una función que reciba dos puntos de $\mathbb{R}^2$ y devuelva distancia y punto medio.
2. Genere veinte vectores aleatorios y conviértalos a forma polar.
3. Compare $\text{atan}(\text{Im}(z)/\text{Re}(z))$ con $\text{Arg}(z)$ en los cuatro cuadrantes y explique las diferencias.
4. Escriba una función que clasifique un complejo como real, imaginario puro o complejo general.
5. Compruebe experimentalmente $|zw| = |z||w|$ con mil pares aleatorios.
6. Represente z , $\bar{z}$, $-z$ y $1/z$ para un mismo complejo no nulo.
7. Calcule $(1 - i)^{20}$ directamente y mediante `potencia_demoivre()`.
8. Obtenga las raíces quintas de -32 y verifique el residuo máximo $\max_k |w_k^5 + 32|$.
9. Dibuje las raíces de la unidad para $n = 3, 4, 5, 8, 12$ en cinco paneles.
10. Modifique el gráfico de raíces para mostrar también los argumentos en grados.

Proyecto integrador

Explorador de transformaciones complejas. Desarrolle un script en R base que:

1. reciba un conjunto de puntos complejos;
2. aplique $T(z) = az + b$ con parámetros elegidos por el usuario;
3. muestre los puntos originales y transformados en la misma escala;
4. separe el efecto de az en rotación y cambio de escala;
5. interprete el efecto de la traslación b ;
6. incluya al menos una figura formada por los vértices de un polígono;
7. compruebe numéricamente dos propiedades del módulo o del conjugado;

8. entregue código, gráfica, explicación matemática y conclusiones.

Síntesis

- Un par de $\mathbb{R}^2$ se almacena como vector; una colección, como tabla.
- `atan2(y, x)` y `Arg(z)` resuelven correctamente el cuadrante.
- R representa complejos con el tipo `complex` y el literal `1i`.
- `Re()`, `Im()`, `Mod()`, `Arg()` y `Conj()` traducen directamente las definiciones.
- La suma es vectorial; el producto combina escala y rotación.
- El conjugado permite dividir y visualizar una reflexión.
- `exp(1i * theta)` simplifica la forma polar, De Moivre y las raíces.
- Las comparaciones numéricas deben usar tolerancia.

Referencias del capítulo

La organización matemática sigue Universidad Autónoma de San Luis Potosí, Facultad de Ciencias (2011) y Silva y Lazo (2007). La interpretación geométrica se apoya en Brown y Churchill (2014) y Needham (1997). Las funciones y la sintaxis computacional corresponden a R base (R Core Team 2026).

4 Polinomios

Este capítulo desarrolla la Unidad 4 del programa oficial de Álgebra Superior de la Facultad de Ciencias de la UASLP (Universidad Autónoma de San Luis Potosí, Facultad de Ciencias 2011). El propósito es pasar de las operaciones formales con polinomios a la comprensión de sus raíces, su factorización y las funciones racionales.

Introducción

Los polinomios aparecen al modelar trayectorias, señales, costos, aproximaciones y ecuaciones. Tienen una doble naturaleza: son expresiones algebraicas que pueden sumarse y multiplicarse, y también funciones cuyos ceros se observan en una gráfica. La división conecta ambas perspectivas: una raíz produce un factor y un factor produce una raíz.

Objetivos del capítulo

Al terminar el capítulo, el lector podrá:

- identificar coeficientes, grado y coeficiente principal;
- efectuar operaciones y justificar sus propiedades;
- aplicar el algoritmo de la división en $K[x]$;
- relacionar raíces, factores y residuos;
- usar división sintética y detectar raíces múltiples;
- explicar el alcance del teorema fundamental del álgebra;
- factorizar sobre $\mathbb{C}$ y sobre $\mathbb{R}$;
- analizar dominio, ceros, discontinuidades y asíntotas de funciones racionales;
- descomponer fracciones racionales en fracciones parciales.

4.1 Definición de polinomio

Sea K un campo, normalmente $\mathbb{Q}$, $\mathbb{R}$ o $\mathbb{C}$.

i Definición

Un polinomio en la variable x con coeficientes en K es una expresión

$$p(x) = a_0 + a_1x + \cdots + a_nx^n, \quad a_j \in K,$$

en la que solo un número finito de coeficientes es distinto de cero. El conjunto de todos ellos se denota por $K[x]$.

Si $p \neq 0$ y $a_n \neq 0$, el **grado** de p es $\deg p = n$, a_n es el coeficiente principal y a_0 el término independiente. Un polinomio es **mónico** cuando su coeficiente principal es 1. Por convenio, al polinomio cero no se le asigna un grado ordinario; a veces se escribe $\deg 0 = -\infty$ para conservar algunas fórmulas.

* Ejemplo

En

$$p(x) = -2x^5 + 3x^2 - 7$$

los coeficientes omitidos son cero, el grado es 5, el coeficiente principal es -2 y el término independiente es -7 .

Igualdad y evaluación

Dos polinomios son iguales si tienen el mismo coeficiente en cada potencia de x . Por ejemplo,

$$(x + 1)^2 = x^2 + 2x + 1$$

como polinomios, porque al desarrollar coinciden todos sus coeficientes.

La evaluación en $c \in K$ es

$$p(c) = a_0 + a_1c + \cdots + a_nc^n.$$

No debe confundirse el polinomio formal con la función que induce. Sobre campos infinitos, como $\mathbb{R}$ y $\mathbb{C}$, dos polinomios que dan el mismo valor para todo x tienen los mismos coeficientes. Sobre un campo finito pueden existir polinomios distintos que representan la misma función.

4.2 Aritmética y propiedades de los polinomios

Para sumar polinomios se suman coeficientes de términos del mismo grado. Para multiplicarlos se aplica la distributividad y se agrupan potencias iguales.

Sean

$$p(x) = \sum_{j=0}^m a_j x^j, \quad q(x) = \sum_{k=0}^n b_k x^k.$$

Entonces

$$p(x) + q(x) = \sum_{r \geq 0} (a_r + b_r) x^r$$

y

$$p(x)q(x) = \sum_{r=0}^{m+n} \left(\sum_{j+k=r} a_j b_k \right) x^r.$$

La suma interior es una **convolución** de las sucesiones de coeficientes.

* Ejemplo resuelto

Si $p(x) = 2x^2 - x + 3$ y $q(x) = x^2 + 4x - 1$, entonces

$$p + q = 3x^2 + 3x + 2,$$

$$pq = 2x^4 + 7x^3 - 3x^2 + 13x - 3.$$

Propiedades de grado

Para polinomios no nulos sobre un campo:

$$\deg(pq) = \deg p + \deg q,$$

$$\deg(p + q) \leq \max\{\deg p, \deg q\}.$$

En la suma puede disminuir el grado si se cancelan los términos principales. Por ejemplo, $(x^3 + 1) + (-x^3 + x) = x + 1$.

Con la suma y el producto, $K[x]$ es un anillo conmutativo con identidad. No es un campo: un polinomio no constante no tiene inverso polinomial. Sus unidades son precisamente las constantes no nulas.

4.3 Divisibilidad

i Definición

Dados $p, d \in K[x]$, con $d \neq 0$, se dice que d divide a p , y se escribe $d \mid p$, si existe $q \in K[x]$ tal que

$$p = dq.$$

El resultado fundamental es análogo al algoritmo de la división de enteros.

! Algoritmo de la división

Para $p, d \in K[x]$, con $d \neq 0$, existen polinomios únicos q y r tales que

$$p = dq + r, \quad r = 0 \quad \text{o} \quad \deg r < \deg d.$$

El polinomio q es el cociente y r el residuo.

Por qué termina el algoritmo

Si $\deg p \geq \deg d$, se elige un monomio que cancele el término principal de p . Después de restar ese múltiplo de d , el grado disminuye. Como los grados son enteros no negativos, el proceso debe terminar.

Ejemplo de división larga

Dividamos

$$p(x) = 2x^4 - 3x^3 + 4x - 5$$

entre $d(x) = x^2 - x + 1$. El resultado es

$$q(x) = 2x^2 - x - 3, \quad r(x) = 2x - 2.$$

La comprobación decisiva es

$$2x^4 - 3x^3 + 4x - 5 = (x^2 - x + 1)(2x^2 - x - 3) + (2x - 2).$$

Algoritmo de la división en $K[x]$

$$\boxed{\text{Dividendo } p(x)} = \boxed{\text{Divisor } d(x) \times \text{cociente } q(x)} + \boxed{\text{Residuo } r(x)}$$

$$p(x) = d(x)q(x) + r(x)$$

$$r(x) = 0 \text{ o } \deg r < \deg d$$

Comprobación: multiplique divisor y cociente, luego sume el residuo.

Figura 4.1: La división polinomial produce un cociente y un residuo de grado menor que el divisor.

4.4 Definición de raíz de un polinomio

i Definición

Un número $\alpha \in K$ es raíz o cero de $p \in K[x]$ si

$$p(\alpha) = 0.$$

La conexión entre evaluación y divisibilidad es inmediata al dividir entre $x - \alpha$.

! Teorema del factor

α es raíz de p si y solo si $x - \alpha$ divide a p .

En efecto, por el algoritmo de la división,

$$p(x) = (x - \alpha)q(x) + r,$$

donde r es constante. Al evaluar en $x = \alpha$ se obtiene $r = p(\alpha)$. Por tanto, $p(\alpha) = 0$ exactamente cuando el residuo es cero.

Número máximo de raíces

Un polinomio no nulo de grado n tiene como máximo n raíces distintas en un campo. La prueba es inductiva: cada raíz extrae un factor lineal y reduce el grado en uno.

! Una distinción esencial

Que un polinomio de grado n tenga a lo sumo n raíces no afirma que tenga n raíces reales. Por ejemplo, $x^2 + 1$ no tiene raíces en $\mathbb{R}$, aunque sí tiene dos en $\mathbb{C}$.

4.5 Teorema del residuo y división sintética

! Teorema del residuo

El residuo de dividir $p(x)$ entre $x - c$ es $p(c)$.

Esto permite evaluar un polinomio sin completar toda la división. La **división sintética** organiza las operaciones solo con coeficientes.

Ejemplo resuelto

Para dividir

$$p(x) = 2x^3 - 3x^2 - 11x + 6$$

entre $x - 3$, se usan los coeficientes $2, -3, -11, 6$:

$$\begin{array}{r|rrrr} 3 & 2 & -3 & -11 & 6 \\ & & 6 & 9 & -6 \\ \hline & 2 & 3 & -2 & 0 \end{array}$$

El cociente es $2x^2 + 3x - 2$ y el residuo es 0. En consecuencia,

$$p(x) = (x - 3)(2x^2 + 3x - 2).$$

! Coeficientes faltantes

En $x^4 - 5x^2 + 4$ deben escribirse los coeficientes 1, 0, -5, 0, 4. Omitir los ceros desplaza las columnas y cambia el resultado.

Evaluación de Horner

La misma organización conduce a

$$p(c) = (((a_n c + a_{n-1})c + a_{n-2})c + \cdots)c + a_0.$$

El esquema de Horner necesita n multiplicaciones y n sumas, y evita calcular por separado todas las potencias de c .

4.6 Obtención de raíces múltiples

i Multiplicidad

Una raíz α tiene multiplicidad $m \geq 1$ si

$$p(x) = (x - \alpha)^m q(x), \quad q(\alpha) \neq 0.$$

La raíz es simple cuando $m = 1$, doble cuando $m = 2$, triple cuando $m = 3$, etcétera.

La derivada permite reconocer multiplicidades:

$$\alpha \text{ tiene multiplicidad al menos } m \iff p(\alpha) = p'(\alpha) = \cdots = p^{(m-1)}(\alpha) = 0.$$

Si además $p^{(m)}(\alpha) \neq 0$, la multiplicidad es exactamente m .

Ejemplo

Sea

$$p(x) = (x - 1)^3(x + 2)^2.$$

La raíz 1 tiene multiplicidad 3 y la raíz -2 multiplicidad 2. En la gráfica, una raíz de multiplicidad par toca el eje y regresa; una raíz de multiplicidad impar lo cruza. Para multiplicidades impares mayores que uno, el cruce se aplana.

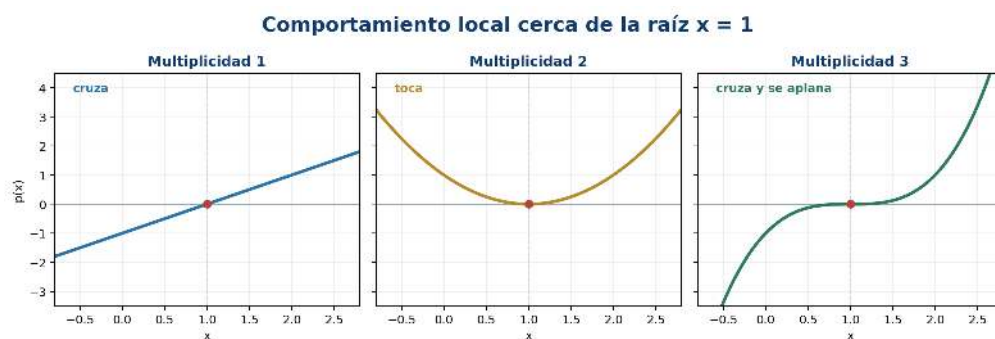


Figura 4.2: Comportamiento local de raíces de multiplicidad uno, dos y tres.

* Procedimiento con divisiones sucesivas

Si $p(\alpha) = 0$, divida entre $x - \alpha$. Vuelva a evaluar el cociente en α y repita. El número de divisiones exactas es la multiplicidad.

4.7 Teorema fundamental del álgebra

! Teorema fundamental del álgebra

Todo polinomio no constante con coeficientes complejos posee al menos una raíz compleja.

Aunque el enunciado garantiza una raíz, al dividir por su factor lineal y repetir se obtiene la forma completa:

$$p(x) = a_n \prod_{j=1}^n (x - \alpha_j),$$

donde las raíces se cuentan con multiplicidad.

Alcance del teorema

- Garantiza existencia, pero no proporciona por sí solo un algoritmo práctico para calcular las raíces.
- El campo $\mathbb{C}$ es algebraicamente cerrado.
- Un polinomio real puede no factorizarse completamente en factores lineales reales, pero sí en factores lineales complejos.
- Las fórmulas generales por radicales existen hasta grado cuatro; para grado cinco o mayor no existe una fórmula por radicales válida para todos los polinomios.

! No es un método numérico

El teorema fundamental del álgebra no dice que todas las raíces puedan escribirse con una fórmula elemental. La aproximación numérica será el tema del capítulo 5.

4.8 Descomposición de un polinomio en factores lineales

Si $p \in \mathbb{C}[x]$ tiene grado $n \geq 1$, entonces

$$p(x) = a_n (x - \alpha_1)^{m_1} \cdots (x - \alpha_s)^{m_s},$$

donde $\alpha_1, \dots, \alpha_s$ son raíces distintas y

$$m_1 + \cdots + m_s = n.$$

Esta factorización es única salvo por el orden de los factores.

Ejemplo completo

Factoricemos

$$p(x) = x^4 - 5x^2 + 4.$$

Al verlo como cuadrático en $u = x^2$:

$$u^2 - 5u + 4 = (u - 1)(u - 4).$$

Por tanto,

$$p(x) = (x^2 - 1)(x^2 - 4) = (x - 1)(x + 1)(x - 2)(x + 2).$$

Las raíces son $-2, -1, 1, 2$, todas simples.

Estrategia para coeficientes enteros

Para un polinomio

$$a_n x^n + \cdots + a_0$$

con coeficientes enteros, toda raíz racional reducida r/s satisface $r \mid a_0$ y $s \mid a_n$. Este **teorema de la raíz racional** produce candidatos; cada candidato debe evaluarse. No asegura que exista una raíz racional.

4.9 Propiedades de polinomios con coeficientes reales

Si p tiene coeficientes reales, la conjugación conmuta con la evaluación:

$$p(\bar{z}) = \overline{p(z)}.$$

Por consiguiente, si $p(z) = 0$, entonces

$$p(\bar{z}) = \bar{0} = 0.$$

! Raíces conjugadas

Las raíces complejas no reales de un polinomio con coeficientes reales aparecen en pares conjugados y con la misma multiplicidad.

El producto de los factores conjugados tiene coeficientes reales:

$$(x - z)(x - \bar{z}) = x^2 - 2 \operatorname{Re}(z)x + |z|^2.$$

Así, todo polinomio real se factoriza sobre $\mathbb{R}$ como producto de factores lineales reales y factores cuadráticos irreducibles.

Ejemplo

Si $2 + i$ es raíz de un polinomio real, también lo es $2 - i$, y

$$[x - (2 + i)][x - (2 - i)] = x^2 - 4x + 5.$$

Raíces conjugadas de un polinomio real

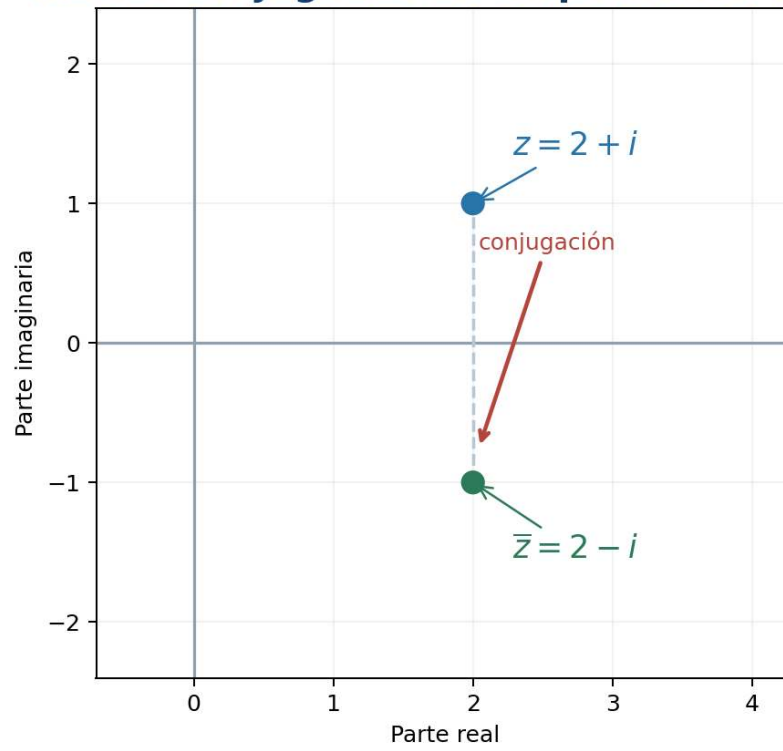


Figura 4.3: Las raíces no reales de un polinomio real son simétricas respecto del eje real.

Relaciones de Viète

Para el polinomio mónico

$$p(x) = x^n + a_{n-1}x^{n-1} + \cdots + a_0 = \prod_{j=1}^n (x - \alpha_j),$$

se cumple, contando multiplicidades,

$$\sum_{j=1}^n \alpha_j = -a_{n-1}, \quad \prod_{j=1}^n \alpha_j = (-1)^n a_0.$$

4.10 Funciones racionales

i Definición

Una función racional es un cociente

$$R(x) = \frac{p(x)}{q(x)},$$

donde $p, q \in K[x]$ y $q \neq 0$. Su dominio excluye los ceros de q .

Antes de simplificar debe registrarse el dominio original. Por ejemplo,

$$R(x) = \frac{x^2 - 1}{x - 1} = x + 1, \quad x \neq 1.$$

La gráfica es la recta $y = x + 1$ con un hueco en $(1, 2)$. La simplificación algebraica no vuelve a incluir $x = 1$.

Ceros, polos y asíntotas

- Los ceros provienen de factores del numerador que no se cancelan.
- Los ceros del denominador no cancelados producen polos y, usualmente, asíntotas verticales.
- Si $\deg p < \deg q$, la asíntota horizontal es $y = 0$.
- Si $\deg p = \deg q$, la asíntota horizontal es el cociente de coeficientes principales.
- Si $\deg p \geq \deg q$, la división polinomial separa una parte polinomial y una fracción propia.

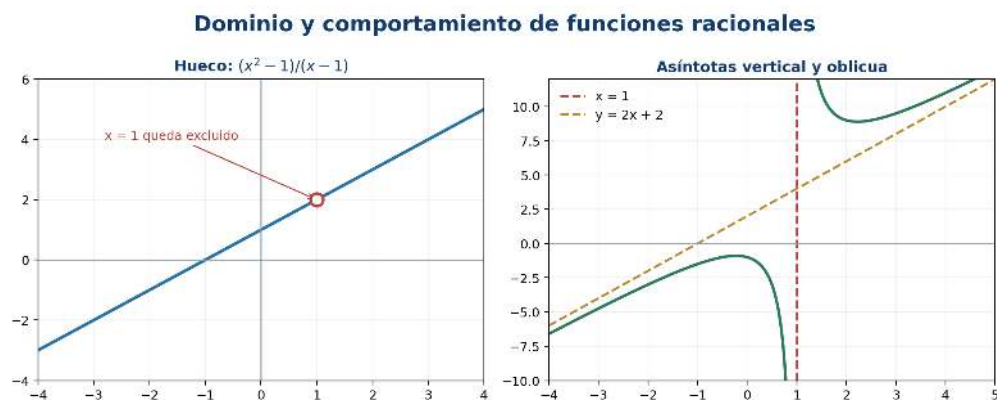


Figura 4.4: Una función racional puede presentar huecos y asíntotas determinadas por sus factores.

* Ejemplo

Para

$$R(x) = \frac{2x^2 + 1}{x - 1},$$

la división da

$$R(x) = 2x + 2 + \frac{3}{x - 1}.$$

Tiene asíntota vertical $x = 1$ y asíntota oblicua $y = 2x + 2$.

4.11 Fracciones parciales

La descomposición en fracciones parciales expresa una fracción racional propia como suma de términos simples. Es útil para integrar, resolver ecuaciones diferenciales y transformar señales.

Primero, si $\deg p \geq \deg q$, se efectúa la división. Después se factoriza el denominador.

Factores lineales distintos

Si

$$\frac{p(x)}{(x-a)(x-b)}$$

es propia y $a \neq b$, se propone

$$\frac{A}{x-a} + \frac{B}{x-b}.$$

Por ejemplo,

$$\frac{5x+1}{(x-1)(x+2)} = \frac{2}{x-1} + \frac{3}{x+2}.$$

La comprobación reúne el lado derecho sobre el denominador común.

Factores lineales repetidos

Para $(x-a)^m$ se requieren todos los términos

$$\frac{A_1}{x-a} + \frac{A_2}{(x-a)^2} + \cdots + \frac{A_m}{(x-a)^m}.$$

Factores cuadráticos irreducibles

Por cada potencia $(x^2+bx+c)^m$ irreducible sobre $\mathbb{R}$ se incluyen numeradores lineales:

$$\frac{A_1x+B_1}{x^2+bx+c} + \cdots + \frac{A_mx+B_m}{(x^2+bx+c)^m}.$$

* Ejemplo mixto

La forma de descomposición para

$$\frac{x^2+1}{(x-1)^2(x^2+4)}$$

es

$$\frac{A}{x-1} + \frac{B}{(x-1)^2} + \frac{Cx+D}{x^2+4}.$$

Los coeficientes se obtienen al multiplicar por el denominador común y comparar coeficientes o evaluar valores convenientes.

Síntesis del capítulo

- $K[x]$ es un anillo de expresiones con un número finito de coeficientes no nulos.
- El algoritmo de la división escribe $p = dq + r$ con $\deg r < \deg d$.
- $p(c)$ es el residuo de dividir por $x - c$.
- c es raíz si y solo si $x - c$ es factor.
- La multiplicidad cuenta cuántas veces aparece el factor lineal.
- Todo polinomio complejo no constante se descompone en factores lineales.
- En polinomios reales, las raíces no reales aparecen en pares conjugados.
- Una función racional conserva las exclusiones de su denominador original.
- Las fracciones parciales dependen de la factorización y multiplicidad del denominador.

Errores frecuentes

1. **Asignar grado al polinomio cero como si fuera ordinario.** Su grado se deja indefinido o se usa $-\infty$ por convenio.
2. **Olvidar términos ausentes.** En división sintética deben colocarse coeficientes cero.
3. **Confundir raíz con factor.** La raíz es un número; el factor correspondiente es $x - \alpha$.
4. **Creer que todo polinomio tiene raíces reales.** La garantía completa corresponde a $\mathbb{C}$.
5. **Contar solo raíces distintas.** El grado coincide con el número de raíces complejas cuando se cuentan multiplicidades.
6. **Cancelar un factor y ampliar el dominio.** Una discontinuidad removible sigue excluida del dominio original.
7. **Omitir términos en fracciones parciales.** Cada potencia repetida necesita su propio término.
8. **Usar una constante sobre un cuadrático irreducible.** El numerador debe tener grado menor, por lo que es lineal.

Ejercicios

Nivel básico

1. Indique grado, coeficiente principal y término independiente de $4x^6 - 3x^2 + 8$.
2. Sume y multiplique $p(x) = x^2 - 2x + 3$ y $q(x) = 2x + 1$.
3. Evalúe $p(x) = 2x^4 - x^2 + 5x - 7$ en $x = -2$ mediante Horner.
4. Divida $x^3 - 2x^2 + 4x - 8$ entre $x - 2$.
5. Determine si -1 es raíz de $x^4 + 3x^3 - x - 3$.
6. Factorice $x^4 - 16$ sobre $\mathbb{R}$ y sobre $\mathbb{C}$.
7. Determine el dominio de $(x^2 - 9)/(x - 3)$ y describa el hueco.

Nivel intermedio

8. Divida $3x^5 - 2x^3 + x - 4$ entre $x^2 + x - 1$ y compruebe el resultado.
9. Use el teorema de la raíz racional para factorizar $2x^3 - x^2 - 8x + 4$.
10. Determine las multiplicidades de las raíces de $x^5 - 2x^4 + x^3$.
11. Construya un polinomio real mónico de grado cuatro cuyas raíces sean 2 , -1 y $3 + i$, $3 - i$.
12. Use Viète para comprobar la suma y el producto de las raíces de $2x^3 - 3x^2 - 11x + 6$.
13. Encuentre ceros, huecos y asíntotas de $(x^2 - 4)/(x^2 - x - 2)$.
14. Descomponga $(7x + 5)/[(x - 1)(x + 3)]$ en fracciones parciales.

Nivel formal y aplicado

15. Demuestre la unicidad del cociente y el residuo en el algoritmo de la división.
16. Pruebe que un polinomio no nulo de grado n tiene a lo sumo n raíces distintas.
17. Demuestre que las raíces conjugadas de un polinomio real tienen la misma multiplicidad.
18. Determine la forma real más general de un polinomio de grado cinco con raíces 1 doble y $2 + 3i$ simple.
19. Descomponga $(2x^3 + x^2 + 4)/[(x - 1)^2(x^2 + 1)]$ en fracciones parciales.
20. Diseñe un polinomio cuya gráfica cruce el eje en $x = -2$, toque el eje en $x = 1$ y tenga grado mínimo.

Proyecto integrador

Elija un polinomio real de grado entre cuatro y seis y presente un estudio completo:

1. identifique grado y coeficientes;
2. localice candidatos racionales;
3. aplique división sintética;
4. determine todas las raíces y multiplicidades;
5. factorice sobre $\mathbb{R}$ y sobre $\mathbb{C}$;
6. compruebe dos relaciones de Viète;
7. construya una función racional usando el polinomio como numerador o denominador;
8. analice dominio, ceros, huecos y asíntotas;
9. obtenga, cuando corresponda, una descomposición en fracciones parciales;
10. acompañe los cálculos con una representación gráfica y conclusiones.

Laboratorio computacional con R y GeoGebra

Este laboratorio traduce las ideas del capítulo a algoritmos reproducibles. Los vectores de coeficientes se almacenan en orden descendente: $a_n, a_{n-1}, \dots, a_0$. Las funciones de R para raíces, en cambio, usan orden ascendente; esta diferencia se señala explícitamente.

Representar y evaluar polinomios en R

```
horner <- function(coeficientes, x) {
  acumulado <- 0
  for (a in coeficientes) acumulado <- acumulado * x + a
  acumulado
}

p <- c(2, -3, 0, 4, -5) # 2x^4 - 3x^3 + 0x^2 + 4x - 5
horner(p, 2)
```

El cero en la tercera posición es indispensable: conserva el lugar del término x^2 .

Suma y producto por coeficientes

```
rellenar_izquierda <- function(a, n) c(rep(0, n - length(a)), a)

sumar_polinomios <- function(a, b) {
```



```

n <- max(length(a), length(b))
rellenar_izquierda(a, n) + rellenar_izquierda(b, n)
}

multiplicar_polinomios <- function(a, b) {
  resultado <- numeric(length(a) + length(b) - 1)
  for (i in seq_along(a)) {
    for (j in seq_along(b)) {
      resultado[i + j - 1] <- resultado[i + j - 1] + a[i] * b[j]
    }
  }
  resultado
}

a <- c(2, -1, 3) # 2x^2 - x + 3
b <- c(1, 4, -1) # x^2 + 4x - 1
sumar_polinomios(a, b)
multiplicar_polinomios(a, b)

```

División polinomial programada

```

quitar_ceros_iniciales <- function(a, tolerancia = 1e-12) {
  while (length(a) > 1 && abs(a[1]) < tolerancia) a <- a[-1]
  a
}

dividir_polinomios <- function(dividendo, divisor, tolerancia = 1e-12) {
  dividendo <- quitar_ceros_iniciales(dividendo, tolerancia)
  divisor <- quitar_ceros_iniciales(divisor, tolerancia)
  if (length(divisor) == 1 && abs(divisor[1]) < tolerancia) {
    stop("El divisor no puede ser el polinomio cero")
  }
  if (length(dividendo) < length(divisor)) {
    return(list(cociente = 0, residuo = dividendo))
  }

  residuo <- dividendo
  cociente <- numeric(length(dividendo) - length(divisor) + 1)

```

```

for (i in seq_along(cociente)) {
  factor <- residuo[i] / divisor[1]
  cociente[i] <- factor
  residuo[i:(i + length(divisor) - 1)] <-
    residuo[i:(i + length(divisor) - 1)] - factor * divisor
}
residuo[abs(residuo) < tolerancia] <- 0
list(cociente = quitar_ceros_iniciales(cociente, tolerancia),
     residuo = quitar_ceros_iniciales(residuo, tolerancia))
}

dividir_polinomios(
  dividendo = c(2, -3, 0, 4, -5),
  divisor = c(1, -1, 1)
)

```

División sintética y multiplicidad

```

division_sintetica <- function(coeficientes, c) {
  salida <- numeric(length(coeficientes))
  salida[1] <- coeficientes[1]
  if (length(coeficientes) > 1) {
    for (j in 2:length(coeficientes)) {
      salida[j] <- coeficientes[j] + c * salida[j - 1]
    }
  }
  list(cociente = salida[-length(salida)], residuo = salida[length(salida)])
}

multiplicidad_raiz <- function(coeficientes, raiz, tolerancia = 1e-9) {
  actual <- coeficientes
  multiplicidad <- 0L
  repeat {
    paso <- division_sintetica(actual, raiz)
    if (abs(paso$residuo) > tolerancia || length(actual) == 1) break
    multiplicidad <- multiplicidad + 1L
    actual <- paso$cociente
  }
}

```

```

multiplicidad
}

p <- c(1, 1, -5, -1, 8, -4) # (x-1)^3 (x+2)^2
multiplicidad_raiz(p, 1)
multiplicidad_raiz(p, -2)

```

Todas las raíces y factorización

La función `polyroot()` recibe coeficientes en orden ascendente. Por eso se aplica `rev()` al vector usado en las funciones anteriores.

```

p_desc <- c(1, 0, -5, 0, 4) # x^4 - 5x^2 + 4
raices <- polyroot(rev(p_desc))
round(raices, 10)

# Verificación numérica mediante Horner
residuos <- vapply(raices, function(z) horner(p_desc, z), complex(1))
max(Mod(residuos))

```

El residuo máximo debe ser cercano a cero. Las pequeñas partes imaginarias que puedan aparecer al resolver un polinomio real son efecto del redondeo numérico.

Pares conjugados

```

z <- 2 + 1i
factor_real <- multiplicar_polinomios(
  c(1, -z),
  c(1, -Conj(z))
)
Re(factor_real) # x^2 - 4x + 5

```

Funciones racionales y fracciones parciales

Para factores lineales distintos, los coeficientes de fracciones parciales pueden obtenerse resolviendo un sistema lineal. En el ejemplo

$$\frac{5x+1}{(x-1)(x+2)} = \frac{A}{x-1} + \frac{B}{x+2},$$

la identidad de numeradores es

$$5x+1 = A(x+2) + B(x-1).$$

```
M <- matrix(c(1, 1,
              2, -1), nrow = 2, byrow = TRUE)
objetivo <- c(5, 1)
coeficientes <- solve(M, objetivo)
names(coeficientes) <- c("A", "B")
coeficientes

# Comprobación en varios puntos del dominio
x <- c(-4, -1, 0, 2, 5)
izquierda <- (5 * x + 1) / ((x - 1) * (x + 2))
derecha <- coeficientes["A"] / (x - 1) + coeficientes["B"] / (x + 2)
max(abs(izquierda - derecha))
```

Cuaderno completo del capítulo 4

[Abrir el cuaderno en Google Colab](#)

[Descargar el cuaderno](#)

El cuaderno reúne las funciones anteriores, ejemplos adicionales, gráficas y una autoevaluación computacional. Todo el código principal usa R base.

Referencias del capítulo

La secuencia temática sigue la Unidad 4 del programa de Álgebra Superior (Universidad Autónoma de San Luis Potosí, Facultad de Ciencias 2011). Las definiciones, el algoritmo de la división, la factorización y las funciones racionales se apoyan en Silva y Lazo (2007), Cárdenas et al. (1999) y Cox et al. (2015). La integración de construcciones dinámicas utiliza la documentación oficial de GeoGebra (GeoGebra 2026b, 2026a).

5 Métodos numéricos para la estimación de raíces

El Teorema Fundamental del Álgebra garantiza la existencia de raíces complejas, pero un problema computacional exige localizar las raíces reales, separarlas y producir aproximaciones verificables. Este capítulo desarrolla las siete secciones de la Unidad 5 del programa de Álgebra Superior de la Facultad de Ciencias de la UASLP (Universidad Autónoma de San Luis Potosí, Facultad de Ciencias 2011). Cada método se construye primero de manera manual y después se implementa con R base.

Introducción

Para

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0, \quad a_n \neq 0,$$

la búsqueda numérica seguirá esta ruta:

1. **acotar:** reducir $\mathbb{R}$ a un intervalo finito;
2. **contar:** determinar cuántas raíces reales hay;
3. **separar:** obtener intervalos con una sola raíz;
4. **aproximar:** aplicar un método iterativo;
5. **validar:** revisar tolerancia, error y residuo.

Objetivos

Al terminar el capítulo, el lector podrá:

- calcular cotas de raíces con R;
- detectar cambios de signo sin omitir coeficientes cero;
- construir una sucesión de Sturm mediante división de polinomios;
- implementar Descartes y usar Budan-Fourier;

- programar bisección, secante y Newton sin paquetes;
- evaluar simultáneamente p y p' con Horner;
- comparar seguridad, velocidad y costo de los métodos;
- experimentar con tres construcciones auténticas de GeoGebra.

Convención computacional

Como en el capítulo 4, un polinomio se representa mediante sus coeficientes en orden descendente. Por ejemplo,

$$x^3 - x - 1$$

se almacena como `c(1, 0, -1, -1)`.

```
normalizar_coef <- function(coef, tol = 0) {
  coef <- as.numeric(coef)
  while (length(coef) > 1L && abs(coef[1L]) <= tol) {
    coef <- coef[-1L]
  }
  coef
}

evaluar_horner <- function(coef, x) {
  coef <- normalizar_coef(coef)
  valor <- rep(coef[1L], length(x))
  if (length(coef) > 1L) {
    for (a in coef[-1L]) valor <- valor * x + a
  }
  valor
}

derivar_polinomio <- function(coef) {
  coef <- normalizar_coef(coef)
  grado <- length(coef) - 1L
  if (grado == 0L) return(0)
  coef[-length(coef)] * seq(grado, 1L)
}
```

5.1 Acotación de raíces

La cota de Cauchy establece que toda raíz z satisface

$$|z| \leq R, \quad R = 1 + \max_{0 \leq k < n} \left| \frac{a_k}{a_n} \right|.$$

Cálculo manual

Para

$$p(x) = x^4 + x^3 - 7x^2 - x + 6,$$

se obtiene

$$R = 1 + \max\{1, 7, 1, 6\} = 8.$$

Por tanto, todas las raíces reales están en $[-8, 8]$.

Implementación en R

```
cota_cauchy <- function(coef) {  
  coef <- normalizar_coef(coef)  
  if (length(coef) == 1L) return(NA_real_)  
  1 + max(abs(coef[-1L] / coef[1L]))  
}  
  
p4 <- c(1, 1, -7, -1, 6)  
R <- cota_cauchy(p4)  
c(-R, R)
```

La cota es segura, pero puede ser amplia. Una tabla de valores dentro de $[-R, R]$ ayuda a localizar subintervalos con cambios de signo.

```

malla <- seq(-R, R, by = 0.5)
tabla_malla <- data.frame(
  x = malla,
  p = evaluar_horner(p4, malla)
)
head(tabla_malla)

```

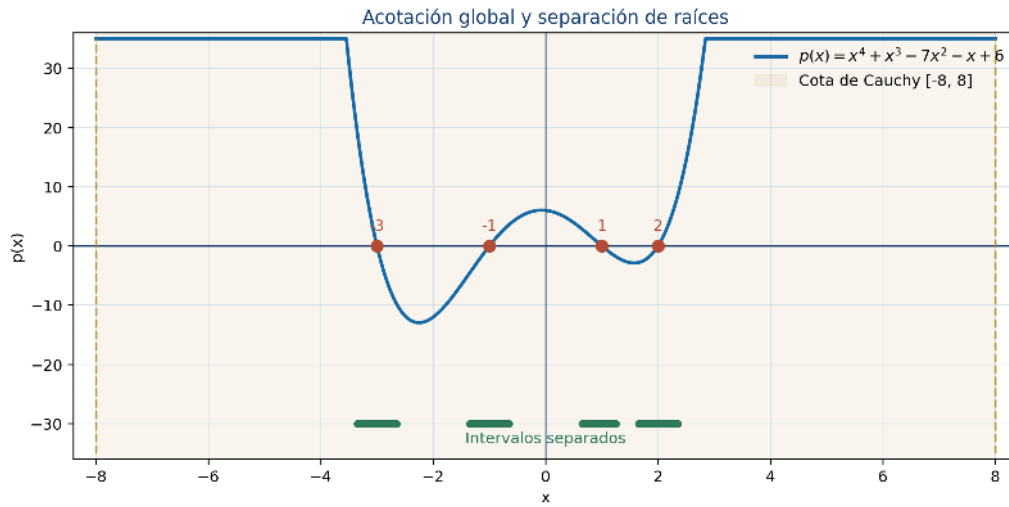


Figura 5.1: La cota global contiene todas las raíces; la malla y los cambios de signo producen intervalos de búsqueda más pequeños.

Actividad GeoGebra

1. Modifique los coeficientes del polinomio desde los deslizadores.
2. Compare la cota $[-R, R]$ con las raíces visibles.
3. Cambie el tamaño de la malla y registre qué intervalos presentan cambio de signo.
4. Construya un ejemplo con una raíz de multiplicidad par y explique por qué no aparece un cambio de signo.

[Descargar la construcción base .ggb](#)

5.2 Separación de raíces

Si $p(a)p(b) < 0$, la continuidad garantiza al menos una raíz en (a, b) . Para detectar automáticamente estos intervalos:

```
separar_por_signo <- function(coef, a, b, paso = 0.25, tol = 1e-12) {  
  x <- seq(a, b, by = paso)  
  if (tail(x, 1L) < b) x <- c(x, b)  
  y <- evaluar_horner(coef, x)  
  
  exactas <- which(abs(y) <= tol)  
  cambios <- which(y[-length(y)] * y[-1L] < 0)  
  
  intervalos <- if (length(cambios)) {  
    data.frame(  
      izquierda = x[cambios],  
      derecha = x[cambios + 1L],  
      f_izquierda = y[cambios],  
      f_derecha = y[cambios + 1L]  
    )  
  } else {  
    data.frame()  
  }  
  
  list(  
    raices_en_malla = x[exactas],  
    intervalos = intervalos  
  )  
}  
  
separar_por_signo(p4, -8, 8, paso = 0.4)
```

Verificar unicidad

Un cambio de signo demuestra existencia, no unicidad. Si p' no cambia de signo y no se anula en $[a, b]$, entonces p es estrictamente monótona y la raíz es única.

Para $f(x) = x^3 - x - 1$ se tiene, exactamente,

$$f'(x) = 3x^2 - 1 \geq 2 \quad \text{para } x \in [1, 2].$$

Por tanto, f es estrictamente creciente y la raíz es única. El siguiente cálculo sólo confirma numéricamente esa cota; el muestreo no sustituye la demostración anterior.

```
fcoef <- c(1, 0, -1, -1)
dfcoef <- derivar_polinomio(fcoef)

c(
  f_1 = evaluar_horner(fcoef, 1),
  f_2 = evaluar_horner(fcoef, 2),
  min_derivada = min(evaluar_horner(dfcoef, seq(1, 2, length.out = 201)))
)
```

El cambio de signo prueba existencia y la desigualdad exacta para la derivada prueba unicidad en $[1, 2]$.

! Limitación del muestreo

Una raíz de multiplicidad par puede tocar el eje sin cambiar de signo. Para no omitirla, se deben analizar p' y el máximo común divisor de p con p' , o usar un conteo de Sturm después de eliminar factores repetidos.

5.3 Teorema de Sturm

La sucesión de Sturm se define por

$$p_0 = p, \quad p_1 = p', \quad p_{k+1} = -\text{residuo}(p_{k-1}, p_k).$$

Si $V(t)$ cuenta las variaciones de signo de los valores de la sucesión en t , el número de raíces reales distintas en (a, b) es $V(a) - V(b)$.

División de polinomios en R base

```
dividir_polinomios <- function(dividendo, divisor, tol = 1e-12) {
  dividendo <- normalizar_coef(dividendo, tol)
  divisor <- normalizar_coef(divisor, tol)
  if (length(divisor) == 1L && abs(divisor) <= tol) {
    stop("El divisor no puede ser el polinomio cero.")
  }
}
```

```

}

grado_a <- length(dividendo) - 1L
grado_b <- length(divisor) - 1L
if (grado_a < grado_b) {
  return(list(cociente = 0, residuo = dividendo))
}

residuo <- dividendo
cociente <- numeric(grado_a - grado_b + 1L)

for (i in seq_along(cociente)) {
  factor <- residuo[i] / divisor[1L]
  cociente[i] <- factor
  residuo[i:(i + grado_b)] <-
    residuo[i:(i + grado_b)] - factor * divisor
}

residuo[abs(residuo) <= tol] <- 0
resto <- if (grado_b == 0L) 0 else tail(residuo, grado_b)
list(
  cociente = normalizar_coef(cociente, tol),
  residuo = normalizar_coef(resto, tol)
)
}

cambios_signo <- function(valores, tol = 1e-10) {
  valores <- valores[abs(valores) > tol]
  if (length(valores) < 2L) return(0L)
  sum(valores[-1L] * valores[-length(valores)]) < 0
}

```

Construcción de la sucesión

```

sucesion_sturm <- function(coef, tol = 1e-10) {
  p0 <- normalizar_coef(coef, tol)
  p1 <- normalizar_coef(derivar_polinomio(p0), tol)
  sucesion <- list(p0, p1)
}

```

```

while (length(sucesion[[length(sucesion)]]) > 1L) {
  n <- length(sucesion)
  r <- dividir_polinomios(sucesion[[n - 1L]], sucesion[[n]], tol)$residuo
  r <- normalizar_coef(-r, tol)
  if (length(r) == 1L && abs(r) <= tol) break
  sucesion[[n + 1L]] <- r
}
sucesion
}

variaciones_sturm <- function(sucesion, x, tol = 1e-10) {
  valores <- vapply(sucesion, evaluar_horner, numeric(1), x = x)
  cambios_signo(valores, tol)
}

contar_raices_sturm <- function(coef, a, b, tol = 1e-10) {
  if (a >= b) stop("Se requiere a < b.")
  if (abs(evaluar_horner(coef, a)) <= tol ||
      abs(evaluar_horner(coef, b)) <= tol) {
    stop("Los extremos del intervalo no deben ser raíces del polinomio.")
  }
  s <- sucesion_sturm(coef, tol)
  variaciones_sturm(s, a, tol) - variaciones_sturm(s, b, tol)
}

sturm_x3_x <- sucesion_sturm(c(1, 0, -1, 0))
sturm_x3_x
contar_raices_sturm(c(1, 0, -1, 0), -2, 2)

```

El resultado es 3, que coincide con las raíces -1 , 0 y 1 . Para raíces múltiples debe trabajarse con la parte libre de cuadrados del polinomio.

Separación certificada

Una estrategia robusta subdivide $[-R, R]$ y conserva los intervalos donde Sturm cuenta exactamente una raíz. A diferencia del simple cambio de signo, este procedimiento también detecta intervalos asociados a raíces pares cuando se ha preparado correctamente el polinomio.

5.4 Regla de los signos de Descartes

Los coeficientes cero se eliminan antes de contar variaciones. El número de raíces positivas es el número de variaciones de $p(x)$ o ese número menos un entero par. Para las negativas se usa $p(-x)$.

```
coef_p_menos_x <- function(coef) {
  coef <- normalizar_coef(coef)
  grado <- length(coef) - 1L
  coef * (-1) ^ seq(grado, 0L)
}

posibilidades_descartes <- function(variaciones) {
  seq(variaciones, 0L, by = -2L)
}

descartes <- function(coef) {
  vp <- cambios_signo(coef)
  vn <- cambios_signo(coef_p_menos_x(coef))
  list(
    variaciones_positivas = vp,
    posibles_positivas = posibilidades_descartes(vp),
    variaciones_negativas = vn,
    posibles_negativas = posibilidades_descartes(vn)
  )
}

descartes(p4)
```

Para p_4 aparecen las posibilidades 2 o 0 en ambos signos. Sturm puede resolver la ambigüedad contando exactamente las raíces en $(-R, 0)$ y $(0, R)$.

```
c(
  negativas = contar_raices_sturm(p4, -R, -1e-8),
  positivas = contar_raices_sturm(p4, 1e-8, R)
)
```

Budan-Fourier

Budan-Fourier cuenta variaciones en

$$p(t), p'(t), \dots, p^{(n)}(t).$$

La diferencia entre las variaciones en a y b es una cota superior para el número de raíces en $(a, b]$ y difiere del número real por un entero par.

```
variaciones_budan <- function(coef, x, tol = 1e-10) {
  valores <- numeric(0)
  actual <- normalizar_coef(coef)
  repeat {
    valores <- c(valores, evaluar_horner(actual, x))
    if (length(actual) == 1L) break
    actual <- derivar_polinomio(actual)
  }
  cambios_signo(valores, tol)
}

cota_budan <- function(coef, a, b) {
  variaciones_budan(coef, a) - variaciones_budan(coef, b)
}

cota_budan(p4, -4, 0)
```

Descartes y Budan-Fourier filtran posibilidades; Sturm certifica el conteo exacto.

5.5 Estimación mediante bisección

En cada paso se calcula el punto medio y se conserva la mitad con cambio de signo.

Algoritmo en R

```
biseccion <- function(coef, a, b, tol = 1e-8, max_iter = 100L) {
  fa <- evaluar_horner(coef, a)
  fb <- evaluar_horner(coef, b)
  if (abs(fa) <= tol) return(data.frame(iter = 0L, a = a, b = a, x = a, fx = fa, error = 0))
  if (abs(fb) <= tol) return(data.frame(iter = 0L, a = b, b = b, x = b, fx = fb, error = 0))
  if (fa * fb > 0) stop("Los extremos deben tener signos opuestos.")
```

```

historial <- vector("list", max_iter)
for (k in seq_len(max_iter)) {
  m <- (a + b) / 2
  fm <- evaluar_horner(coef, m)
  error <- (b - a) / 2
  historial[[k]] <- data.frame(iter = k, a = a, b = b, x = m, fx = fm, error = error)

  if (abs(fm) <= tol || error <= tol) break
  if (fa * fm < 0) {
    b <- m
    fb <- fm
  } else {
    a <- m
    fa <- fm
  }
}
do.call(rbind, historial[seq_len(k)])
}

hist_bis <- biseccion(fcoef, 1, 2, tol = 1e-8)
head(hist_bis, 6)
tail(hist_bis, 1)

```

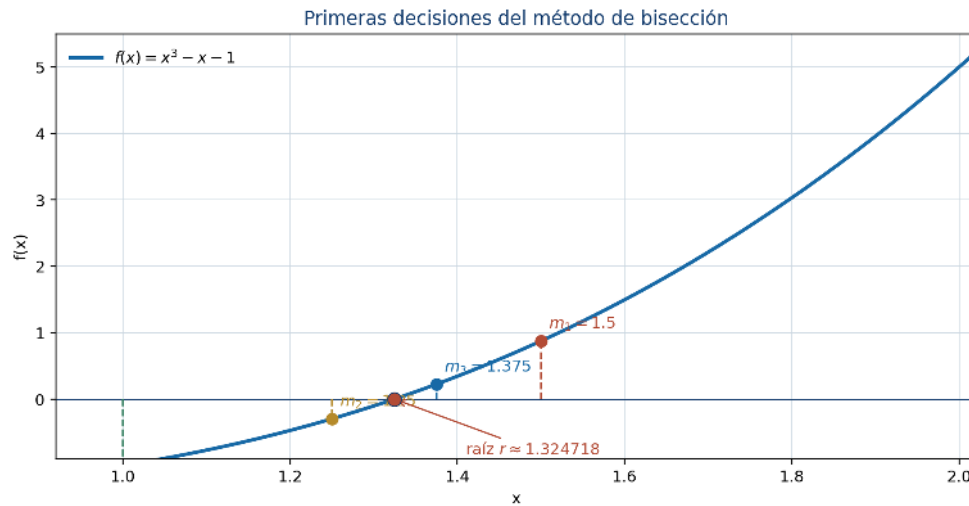


Figura 5.2: Las primeras bisecciones reducen sistemáticamente el intervalo que encierra la raíz.

Actividad GeoGebra

1. Inicie con $[1, 2]$ para $x^3 - x - 1$.
2. Avance una iteración a la vez y anote el intervalo conservado.
3. Compare la longitud visual con $(b_0 - a_0)/2^n$.
4. Descargue el archivo `.ggb` y cambie el polinomio.

[Descargar la construcción base .ggb](#)

5.6 Estimación mediante el método de la secante

La fórmula es

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}.$$

Implementación


```

secante <- function(coef, x0, x1, tol = 1e-8, max_iter = 100L) {
  f0 <- evaluar_horner(coef, x0)
  f1 <- evaluar_horner(coef, x1)
  historial <- vector("list", max_iter)

  for (k in seq_len(max_iter)) {
    denominador <- f1 - f0
    if (abs(denominador) <= .Machine$double.eps) {
      stop("La pendiente de la secante es numéricamente nula.")
    }
    x2 <- x1 - f1 * (x1 - x0) / denominador
    f2 <- evaluar_horner(coef, x2)
    cambio <- abs(x2 - x1)
    historial[[k]] <- data.frame(iter = k, x = x2, fx = f2, cambio = cambio)

    if (abs(f2) <= tol || cambio <= tol) break
    x0 <- x1
    f0 <- f1
    x1 <- x2
    f1 <- f2
  }
  do.call(rbind, historial[seq_len(k)])
}

hist_sec <- secante(fcoef, 1, 2, tol = 1e-10)
hist_sec

```

La secante suele necesitar menos iteraciones que la bisección y no utiliza derivadas. A cambio, no conserva un intervalo seguro y puede fallar si los valores funcionales son casi iguales.

5.7 Estimación mediante el método de Newton

Newton usa la tangente:

$$x_{n+1} = x_n - \frac{p(x_n)}{p'(x_n)}.$$

Horner simultáneo para p y p'

```
horner_valor_derivada <- function(coef, x) {
  coef <- normalizar_coef(coef)
  grado <- length(coef) - 1L
  if (grado == 0L) return(c(valor = coef, derivada = 0))

  b <- coef[1L]
  caux <- b
  if (grado > 1L) {
    for (i in 2L:grado) {
      b <- coef[i] + x * b
      caux <- b + x * caux
    }
  }
  b <- coef[length(coef)] + x * b
  c(valor = b, derivada = caux)
}

horner_valor_derivada(fcoef, 1.5)
```

Newton-Horner en R

```
newton_horner <- function(coef, x0, tol = 1e-8, max_iter = 100L, multiplicidad = 1) {
  historial <- vector("list", max_iter)

  for (k in seq_len(max_iter)) {
    vd <- horner_valor_derivada(coef, x0)
    if (abs(vd["derivada"]) <= .Machine$double.eps) {
      stop("La derivada es numéricamente nula.")
    }
    x1 <- x0 - multiplicidad * vd["valor"] / vd["derivada"]
    f1 <- evaluar_horner(coef, x1)
    cambio <- abs(x1 - x0)
    historial[[k]] <- data.frame(iter = k, x = x1, fx = f1, cambio = cambio)

    if (abs(f1) <= tol || cambio <= tol) break
    x0 <- x1
  }
}
```

```

}
do.call(rbind, historial[seq_len(k)])
}

hist_newton <- newton_horner(fcoef, 1.5, tol = 1e-12)
hist_newton

```

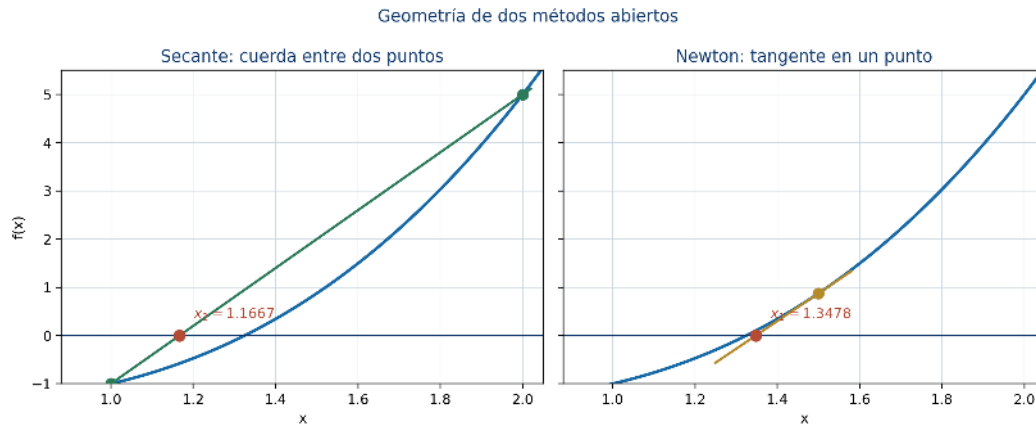


Figura 5.3: La secante usa una cuerda definida por dos puntos; Newton usa una tangente y requiere la derivada.

Actividad GeoGebra

1. Desplace los puntos iniciales de la secante y observe su intersección con el eje.
2. Desplace el punto inicial de Newton y compare la tangente.
3. Localice una posición donde la derivada sea pequeña.
4. Explique por qué la construcción predice un paso de Newton muy grande.

[Descargar la construcción base .ggb](#)

Raíz múltiple

Para $p(x) = (x - 1)^4$, Newton ordinario converge lentamente. Si se conoce $m = 4$, use:

```

raiz_multiple <- c(1, -4, 6, -4, 1)
newton_horner(raiz_multiple, 2, tol = 1e-12, multiplicidad = 1)
newton_horner(raiz_multiple, 2, tol = 1e-12, multiplicidad = 4)

```

La versión modificada incorpora la multiplicidad y puede recuperar convergencia cuadrática.

Comparación reproducible

```
resumen_metodos <- data.frame(
  metodo = c("Bisección", "Secante", "Newton-Horner"),
  iteraciones = c(nrow(hist_bis), nrow(hist_sec), nrow(hist_newton)),
  aproximacion = c(tail(hist_bis$x, 1), tail(hist_sec$x, 1), tail(hist_newton$x, 1)),
  residuo = abs(c(tail(hist_bis$fx, 1), tail(hist_sec$fx, 1), tail(hist_newton$fx, 1)))
)
resumen_metodos
```

Método	Requisito	Fortaleza	Riesgo
Bisección	intervalo con cambio de signo	garantía y cota de error	convergencia lineal
Secante	dos aproximaciones	evita derivadas	denominador pequeño o alejamiento
Newton-Horner	punto inicial y derivada	convergencia cuadrática local	derivada pequeña o inicio inadecuado

Laboratorio del capítulo

Trabaje con

$$p(x) = x^5 - 3x^4 - 3x^3 + 11x^2 - 6x.$$

1. Calcule la cota de Cauchy.
2. Use Descartes para las posibilidades positivas y negativas.
3. Aplique Sturm para contar raíces en $(-R, 0)$ y $(0, R)$.
4. Separe cada raíz en un intervalo.
5. Aproxime cada una con bisección, secante y Newton-Horner.
6. Compare iteraciones y residuos con una misma tolerancia.
7. Represente el polinomio en la construcción GeoGebra de localización.
8. Documente un caso donde un método abierto necesite cambiar sus datos iniciales.

Errores frecuentes

- Omitir coeficientes cero y alterar el grado del polinomio.
- Usar una cota como si fuera una raíz aproximada.
- Concluir unicidad a partir de un cambio de signo.
- Suponer que una malla fina detecta todas las raíces.
- Contar ceros como cambios de signo en Descartes o Sturm.
- Aplicar bisección sin comprobar signos opuestos.
- Ignorar un denominador casi nulo en secante.
- Continuar Newton cuando la derivada es demasiado pequeña.
- Confundir un residuo pequeño con una cota automática del error;
- comparar métodos con tolerancias o criterios de paro diferentes.

Ejercicios graduados

Nivel 1

1. Use `cota_cauchy()` con $2x^4 - 3x^2 + 7x - 5$.
2. Verifique las posibilidades de Descartes para $x^5 - 2x^4 + x^2 - 3$.
3. Ejecute seis iteraciones de bisección para $x^3 - 2$ en $[1, 2]$.
4. Grafique el historial de error de bisección en escala logarítmica.

Nivel 2

5. Use `sucesion_sturm()` para $x^3 - x$ y explique cada polinomio de la lista.
6. Separe las cuatro raíces de $x^4 - 5x^2 + 4$.
7. Compare secante y Newton-Horner para $x^3 - x - 1$ con tolerancia 10^{-10} .
8. Modifique `biseccion()` para registrar el número de evaluaciones funcionales.

Nivel 3

9. Programe una rutina que subdivida $[-R, R]$ hasta obtener intervalos con conteo de Sturm igual a uno.
10. Implemente Newton con protección: si el paso sale del intervalo, use una bisección.
11. Estudie la convergencia de Newton ordinario y modificado para $(x - 1)^m$, con $m = 2, 3, 4$.
12. Diseñe una prueba automática que compare las raíces encontradas con `polyroot()` sin usarlo como justificación teórica.

Proyecto integrador

Elija un polinomio real de grado entre 4 y 7 con al menos tres raíces reales. Construya un informe reproducible que incluya cota, Descartes, Sturm, separación, bisección, secante, Newton-Horner, tablas de iteraciones, gráfica del error y una exploración GeoGebra. Todas las funciones deben usar R base y todos los resultados numéricos deben acompañarse de tolerancia y residuo.

Síntesis

La estimación confiable de raíces es un proceso, no una fórmula única. Cauchy y la malla delimitan la búsqueda; Descartes y Budan-Fourier reducen posibilidades; Sturm cuenta; bisección aporta seguridad; secante evita derivadas; Newton-Horner aprovecha la estructura polinómica para lograr rapidez. R permite conservar todo el historial de cálculo y GeoGebra hace visible la geometría de cada iteración.

Referencias del capítulo

La secuencia temática sigue Universidad Autónoma de San Luis Potosí, Facultad de Ciencias (2011). La teoría de raíces y polinomios se apoya en Cárdenas et al. (1999) y Kurosh (1987). Los algoritmos y criterios numéricos siguen Burden y Faires (2011) y Chapra y Canale (2015). El código utiliza R base (R Core Team 2026) y las construcciones web emplean la incrustación y la API oficiales de GeoGebra (GeoGebra 2026b, 2026a).

Glosario

Algoritmo. Secuencia finita y ordenada de instrucciones para resolver un problema.

Argumento principal. Ángulo elegido en un intervalo convencional, normalmente $(-\pi, \pi]$, para describir la dirección de un número complejo no nulo.

Bisección. Método que conserva repetidamente la mitad de un intervalo donde una función continua cambia de signo.

Biyectiva. Función simultáneamente inyectiva y suprayectiva.

Cardinalidad. Número de elementos de un conjunto finito o medida de tamaño para conjuntos más generales.

Codominio. Conjunto declarado de valores de salida posibles de una función.

Complejo conjugado. Para $z = a + bi$, número $\bar{z} = a - bi$ obtenido al cambiar el signo de la parte imaginaria.

Conjunto. Colección bien definida de objetos llamados elementos.

Construcción dinámica. Recurso de GeoGebra cuyos objetos dependen de parámetros que pueden modificarse interactivamente.

Cota de Cauchy. Límite superior que encierra, en módulo, todas las raíces de un polinomio.

Cuaderno. Documento ejecutable que combina explicación, fórmulas, código y resultados.

Divisibilidad. Relación $a \mid b$ que indica que existe un entero k tal que $b = ak$.

Dominio. Conjunto de entradas para las que se define una función.

Factorización. Escritura de una expresión, entero o polinomio como producto de factores más simples.

Función. Correspondencia que asigna a cada elemento del dominio exactamente una imagen.

Horner, método de. Esquema anidado para evaluar un polinomio y, en algunas variantes, su derivada con menos operaciones.

Inyectiva. Función en la que entradas distintas tienen imágenes distintas.

Iteración. Repetición de un cálculo que produce aproximaciones sucesivas.

Módulo. Distancia de un número complejo al origen; para $z = a + bi$, $|z| = \sqrt{a^2 + b^2}$.

Multiplicidad. Número de veces que un factor $x - \alpha$ aparece en la factorización de un polinomio.

Método de Newton. Iteración que usa la tangente, $x_{n+1} = x_n - f(x_n)/f'(x_n)$, para aproximar una raíz.

Número complejo. Número de la forma $a + bi$, donde $a, b \in \mathbb{R}$ e $i^2 = -1$.

Objeto de R. Estructura identificada por un nombre que almacena datos, funciones o resultados.

Polinomio. Suma finita de términos con potencias enteras no negativas de una variable.

Raíz. Valor que hace cero una función o un polinomio.

Relación. Subconjunto de un producto cartesiano.

Reproducibilidad. Posibilidad de repetir un análisis con los mismos datos, instrucciones y parámetros.

Residuo. Resto de una división entera o polinomial; en aproximación numérica también puede significar el valor $|f(x)|$.

Secante, método de la. Iteración que aproxima una raíz mediante la recta que pasa por dos puntos de la gráfica.

Sucesión de Sturm. Lista de polinomios construida mediante residuos que permite contar exactamente raíces reales distintas en un intervalo.

Suprayectiva. Función cuya imagen coincide con todo el codominio.

Tolerancia. Umbral usado como criterio de parada en un método numérico.

Visualización dinámica. Representación gráfica que permite modificar parámetros y observar inmediatamente sus efectos.

Índices de consulta

Los índices permiten localizar conceptos, código, figuras y laboratorios sin aumentar la numeración principal del libro.

Índice temático

Tabla 5.1: Índice alfabético de conceptos, código y laboratorios.

Término o recurso	Sección principal
Algoritmo de Euclides	2.3 Divisibilidad
Aritmética de punto flotante	2.5 Racionales y reales
Bézout, coeficientes de	2.3 Divisibilidad
Cardinalidad	1.5 Conjuntos finitos y 1.9 Conjuntos infinitos
Circuitos lógicos	aplicación integradora del capítulo 1
Complejidad	2.6 Propiedades de los reales
Conjunto potencia	1.1 Lenguaje de conjuntos
Cuantificadores en dominios finitos	1.3 Cuantificadores
Densidad	2.6 Propiedades de los reales
Divisibilidad	2.3 Divisibilidad
Exponentes	2.7 Exponentes
Factorización prima	2.4 Primos y factorización
Funciones en $\mathbb{R}$	1.7 Funciones
Inducción y verificación	2.2 Inducción
Interactivo de Euclides	2.3 Divisibilidad
Interactivo de funciones	1.8 Tipos de funciones
Interactivo de inducción	2.2 Inducción
Interactivo de tablas de verdad	1.2 Proposiciones
Interactivo de valor absoluto	2.8 Valor absoluto
Interactivo de Venn	1.4 Operaciones de conjuntos
Máximo común divisor	2.3 Divisibilidad
Mínimo común múltiplo	2.4 Primos y factorización

Término o recurso	Sección principal
Números primos	2.4 Primos y factorización
Operaciones con conjuntos en R	1.4 Operaciones de conjuntos
Racionales exactos y aproximados	2.5 Racionales y reales
Relaciones en R	1.6 Producto y relaciones
Valor absoluto	2.8 Valor absoluto
Argumento principal	3.4 Módulo y argumento
Complejo conjugado	3.7 Conjugado
Conversión cartesiana-polar	3.2 Representaciones
De Moivre en R	3.9 Potencias y raíces
Multiplicación como rotación	3.6 Operaciones complejas
Raíces de la unidad	3.9 Potencias y raíces
División sintética	4.5 Residuo y división sintética
Factorización lineal	4.8 Factores lineales
Fracciones parciales	4.11 Fracciones parciales
Función racional	4.10 Funciones racionales
GeoGebra para polinomios	laboratorio computacional del capítulo 4
Multiplicidad de una raíz	4.6 Raíces múltiples
Polinomio	4.1 Definición
Raíces conjugadas	4.9 Polinomios reales
Teorema fundamental del álgebra	4.7 Teorema fundamental
Bisección en R	5.5 Estimación mediante bisección
Cota de Cauchy	5.1 Acotación de raíces
Newton-Horner	5.7 Estimación mediante Newton
Regla de Descartes	5.4 Regla de los signos de Descartes
Secante en R	5.6 Estimación mediante secante
Separación de raíces	5.2 Separación de raíces
Teorema de Sturm	5.3 Teorema de Sturm

Índice de funciones y operadores de R

Tabla 5.2: Funciones y operadores principales de R.

Función u operador	Uso matemático	Sección
<code>%in%</code>	pertenencia	1.1
<code>union()</code> , <code>intersect()</code> , <code>setdiff()</code>	operaciones de conjuntos	1.4

Función u operador	Uso matemático	Sección
<code>expand.grid()</code>	tablas de verdad y productos cartesianos	1.2 y 1.6
<code>all()</code> , <code>any()</code>	cuantificadores en dominios finitos	1.3
<code>%/%</code> , <code>%%</code>	cociente y residuo	2.3
<code>euclides()</code>	máximo común divisor	2.3
<code>euclides_extendido()</code>	identidad de Bézout	2.3
<code>es_primo()</code>	prueba de primalidad	2.4
<code>criba()</code>	primos hasta un límite	2.4
<code>factorizar()</code>	factores primos	2.4
<code>all.equal()</code>	comparación numérica aproximada	2.5 y 2.7
<code>abs()</code>	valor absoluto y distancia	2.8
<code>complex()</code>	construir complejos desde componentes	3.5
<code>Re()</code> , <code>Im()</code>	partes real e imaginaria	3.5
<code>Mod()</code> , <code>Arg()</code>	módulo y argumento principal	3.4
<code>Conj()</code>	complejo conjugado	3.7
<code>exp()</code>	forma polar, potencias y raíces	3.6 y 3.9
<code>horner()</code>	evaluación eficiente de polinomios	4.5 y laboratorio del capítulo 4
<code>dividir_polinomios()</code>	cociente y residuo polinomiales	laboratorio del capítulo 4
<code>division_sintetica()</code>	división entre $x - c$	laboratorio del capítulo 4
<code>multiplicidad_raiz()</code>	multiplicidad por divisiones sucesivas	laboratorio del capítulo 4
<code>polyroot()</code>	aproximación de todas las raíces complejas	laboratorio del capítulo 4
<code>solve()</code>	coeficientes de fracciones parciales	laboratorio del capítulo 4
<code>cota_cauchy()</code>	acotar todas las raíces	5.1
<code>separar_por_signo()</code>	localizar intervalos con cambio de signo	5.2
<code>sucesion_sturm()</code>	construir la sucesión de Sturm	5.3
<code>contar_raices_sturm()</code>	contar raíces distintas en un intervalo	5.3

Función u operador	Uso matemático	Sección
<code>descartes()</code>	posibilidades de raíces positivas y negativas	5.4
<code>biseccion()</code>	aproximación cerrada con cota de error	5.5
<code>secante()</code>	aproximación sin derivadas	5.6
<code>horner_valor_derivada()</code>	evaluar p y p' simultáneamente	5.7
<code>newton_horner()</code>	aproximación rápida de raíces	5.7

Índice de símbolos

Tabla 5.3: Símbolos matemáticos principales.

Símbolo	Significado	Sección
$x \in A, A \subseteq B$	pertenencia e inclusión	1.1
$\neg, \wedge, \vee$	conectivos lógicos	1.2
$\forall, \exists$	cuantificadores	1.3
$A \cup B, A \cap B, A^c$	operaciones de conjuntos	1.4
$f : A \rightarrow B$	función	1.7
$\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}$	sistemas numéricos	2.1 y 2.5
$a \mid b$	divisibilidad	2.3
$\gcd(a, b)$	máximo común divisor	2.3
$\text{mcm}(a, b)$	mínimo común múltiplo	2.4
$ x - a $	distancia entre x y a	2.8
$\mathbb{R}^2, \mathbb{C}$	plano real y números complejos	3.1 y 3.5
$i^2 = -1$	unidad imaginaria	3.5
$\bar{z}$	conjugado	3.7
$ z , \arg(z)$	módulo y argumento	3.4
$e^{i\theta}$	forma exponencial compleja	3.9
$K[x]$	anillo de polinomios con coeficientes en K	4.1
$\deg p$	grado del polinomio p	4.1

Símbolo	Significado	Sección
$p = dq + r$	identidad de la división polinomial	4.3
$p(\alpha) = 0$	α es raíz de p	4.4
R	cota global de las raíces	5.1
$V(t)$	variaciones de signo	5.3
ε	tolerancia numérica	5.5 a 5.7
$f'(x)$	derivada utilizada por Newton	5.7

Índice de figuras

Capítulo 1

1. Operaciones y regiones de Venn (sección 1.4).
2. Relación, función y no-función (sección 1.7).
3. Composición de funciones (sección 1.7).
4. Funciones inyectivas, suprayectivas y biyectivas (sección 1.8).
5. Circuito para $X = (A + \overline{B})C$ (aplicación integradora).
6. Diagrama de temporización del circuito (aplicación integradora).

Capítulo 2

1. Suma de impares y cuadrados perfectos (sección 2.2).
2. Residuos del algoritmo de Euclides (sección 2.3).
3. Cadena de factorización de 756 (sección 2.4).
4. Jerarquía de los sistemas numéricos (sección 2.5).
5. Valor absoluto como distancia (sección 2.8).

Capítulo 3

1. Plano complejo y coordenadas polares (sección 3.2).
2. Suma vectorial y regla del paralelogramo (sección 3.3).
3. Conjugación como reflexión (sección 3.7).
4. Raíces de la unidad y polígonos regulares (sección 3.9).

Capítulo 4

1. División polinomial: dividendo, divisor, cociente y residuo (sección 4.3).
2. Multiplicidad: raíces simples, dobles y triples (sección 4.6).
3. Raíces conjugadas: simetría respecto del eje real (sección 4.9).
4. Funciones racionales: huecos y asíntotas (sección 4.10).

Capítulo 5

1. Acotación global y separación de raíces (secciones 5.1 y 5.2).
2. Primeras iteraciones de bisección (sección 5.5).
3. Comparación geométrica de secante y Newton (secciones 5.6 y 5.7).

Índice de laboratorios

Tabla 5.4: Laboratorios interactivos incorporados.

Laboratorio	Concepto	Sección
Tablas de verdad	conectivos y clasificación	1.2
Operaciones de Venn	unión, intersección y complemento	1.4
Clasificador de funciones	inyectividad, suprayectividad y biyección	1.8
Inducción visual	suma de impares	2.2
Algoritmo de Euclides	cocientes, residuos y MCD	2.3
Valor absoluto	distancia e intervalos	2.8
Plano complejo	coordenadas cartesianas, módulo y argumento	3.2
Multiplicación compleja	cambio de escala y rotación	3.6
Raíces de la unidad	ángulos y polígonos regulares	3.9
Operaciones con polinomios	suma, producto y coeficientes	laboratorio del capítulo 4
División y residuo	teoremas del residuo y del factor	laboratorio del capítulo 4
Raíces y multiplicidad	cruce, contacto y aplanamiento	laboratorio del capítulo 4
Factorización lineal	raíces y factores	laboratorio del capítulo 4

Laboratorio	Concepto	Sección
Raíces conjugadas	simetría en el plano complejo	laboratorio del capítulo 4
Funciones racionales	huecos y asíntotas	laboratorio del capítulo 4
GeoGebra: localización de raíces	cota de Cauchy e intersecciones	5.1
GeoGebra: bisección	intervalo, punto medio y error	5.5
GeoGebra: secante y Newton	cuerda, tangente y siguiente aproximación	5.6 y 5.7

Referencias generales

Las fuentes de todos los capítulos se reúnen aquí en una lista única. Las notas bibliográficas al final de cada capítulo indican cuáles sustentan cada bloque temático.

Brown, James Ward, y Ruel V. Churchill. 2014. *Complex Variables and Applications*. 9.^a ed. McGraw-Hill Education.

Burden, Richard L., y J. Douglas Faires. 2011. *Numerical Analysis*. 9.^a ed. Brooks/Cole, Cengage Learning.

Burton, David M. 2011. *Elementary Number Theory*. 7.^a ed. McGraw-Hill.

Cárdenas, Humberto, Emilio Lluís, Francisco Raggi, y Francisco Tomás. 1999. *Álgebra superior*. 2.^a ed. Trillas.

Chapra, Steven C., y Raymond P. Canale. 2015. *Numerical Methods for Engineers*. 7.^a ed. McGraw-Hill Education.

Cox, David A., John Little, y Donal O'Shea. 2015. *Ideals, Varieties, and Algorithms*. 4.^a ed. Springer.

Epp, Susanna S. 2011. *Discrete Mathematics with Applications*. 4.^a ed. Brooks/Cole, Cengage Learning.

GeoGebra. 2026a. «GeoGebra Apps API». https://geogebra.github.io/docs/reference/en/GeoGebra_Apps_API/.

GeoGebra. 2026b. «GeoGebra Apps Embedding». https://geogebra.github.io/docs/reference/en/GeoGebra_Apps_Embedding/.

Kurosh, Aleksandr G. 1987. *Curso de álgebra superior*. Editorial Mir.

Needham, Tristan. 1997. *Visual Complex Analysis*. Oxford University Press.

- Niven, Ivan, Herbert S. Zuckerman, y Hugh L. Montgomery. 1991. *An Introduction to the Theory of Numbers*. 5.^a ed. John Wiley & Sons.
- R Core Team. 2026. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing.
- Rosen, Kenneth H. 2019. *Discrete Mathematics and Its Applications*. 8.^a ed. McGraw-Hill Education.
- Silva, Juan Manuel, y Adriana Lazo. 2007. *Fundamentos de Matemáticas: Álgebra, Trigonometría, Geometría Analítica y Cálculo*. 7.^a ed. Limusa.
- Tocci, Ronald J., y Neal S. Widmer. 2003. *Sistemas digitales: principios y aplicaciones*. 8.^a ed. Pearson Educación.
- Universidad Autónoma de San Luis Potosí, Facultad de Ciencias. 2011. *Programa sintético de Álgebra Superior, carrera de Ingeniero Electrónico*. Propuesta de modificación curricular.

Acerca del autor

Jesús Gilberto Rodríguez Escobedo es Licenciado en Físico-Matemáticas, Maestro en Ciencias de la Computación y Maestro en Inteligencia Artificial. Cuenta con cerca de 45 años de experiencia en docencia universitaria en matemáticas, estadística, computación, ciencia de datos y aprendizaje automático.

Su labor académica se ha desarrollado principalmente en la Universidad Autónoma de San Luis Potosí, en las facultades de Ciencias e Ingeniería. Sus intereses incluyen la enseñanza apoyada en tecnología, la informática educativa, los sistemas tutoriales inteligentes, el análisis de datos y la inteligencia artificial.

En sus proyectos editoriales busca vincular el razonamiento manual con recursos interactivos y computacionales, de modo que estudiantes y profesores puedan estudiar, experimentar y reutilizar materiales abiertos.